CrewGuard

Regulatory compliance validation and violation explanation for airline crew rosters

Final project report

DBMS Experiential Learning, Level 3 · Group G22 · November 2026

Member	SRN	Part
Samyak Sanklecha	PES2UG24CS436	Part 1: roster and platform
Shriram Chandrasekar	PES2UG24CS495	Part 2: compliance and analytics

Contents

1. Introduction 4
1.1 Problem statement 4
1.2 Objectives 4
1.3 Scope 4
2. Functional requirements 4
3. Database design 5
3.1 ER diagram 5
3.2 Relational schema 6
3.3 Normalisation 6
3.4 Database choice and justification 6
4. Data dictionary 7
5. SQL implementation 11
5.1 DDL and constraints 11
5.2 ALTER: the review workflow 11
5.3 Views 11
5.4 Functions 12
5.5 Triggers 12
5.6 Stored procedure: the validator 13
5.7 Indexes 15
5.8 Transactions 16
5.9 Full-text search 16
6. NoSQL implementation 16
6.1 Collections 16
6.2 Sample documents 16
6.3 Aggregation pipelines 18
6.4 Writing evidence, and what happens when MongoDB is down 19
7. Important queries 19
8. System architecture 25
9. Application screenshots 25
9.1 Signing in 26
9.2 Scheduler: the roster 26
9.3 The database rejecting bad input 33
9.4 Auditor: runs and violations 36
9.5 The explanation pages 39

9.6 Rules and analytics....44
10. Testing....46
10.1 What the tests prove....46
10.2 Output of npm test....46
10.3 Expected validator results....47
10.4 End-to-end checks in the running app....47
10.5 Tested against a real MongoDB....48
11. Conclusion and future enhancements....48
Known limitations....48
Future enhancements....48
12. Individual contributions....49

1. Introduction

1.1 Problem statement

An airline roster can be fully staffed and still be illegal. Aviation regulators limit how long crew may work and how much rest they must get, and most of those limits depend on time windows across many duties, not on any single flight. A rule like "no more than 60 duty hours in any 7 consecutive days" cannot be checked by looking at one flight at a time, and every duty in such a roster can look perfectly fine on its own.

CrewGuard is a detector and an explainer, not a roster generator. A scheduler builds the roster, then an auditor runs a validator over a date range. The validator records every breach and links it to the exact duties that caused it, so the auditor can see why something is illegal and not just that it is.

1.2 Objectives

- Store the roster in PostgreSQL with the integrity rules enforced by the database itself.
- Detect breaches of four rules with a stored procedure: minimum rest, maximum duty length, maximum hours in a rolling 7 days, and valid type ratings.
- Explain each breach by linking it to the duties that caused it, through a many-to-many table.
- Give auditors a review workflow with an accountable trail of who did what.
- Keep rule-specific evidence and an activity log in MongoDB, and use aggregation pipelines for analytics.
- Separate prevention (the database rejects the row) from detection (the validator reports a breach on legal-looking data).

1.3 Scope

Rule	What it checks	Threshold	Severity	How it is handled
RST	Rest between two consecutive duties	at least 10 h	HIGH	Detected: `LAG()` window function
FDP	Length of a single duty	at most 13 h	CRITICAL	Detected: duty length against the limit
WDH	Duty hours in any rolling 7 days	at most 60 h	MEDIUM	Detected: rolling `SUM() OVER (RANGE ...)`
CER	Pilot holds a valid rating for the aircraft flown (cabin crew exempt)	none	CRITICAL	Detected: anti-join with `NOT EXISTS`
none	No overlapping duties for one crew member	none	none	Prevented: `EXCLUDE USING GIST` constraint

Out of scope: generating or optimising rosters, live flight operations data, and the full regulation text. The sample data is a two-week roster from 2 to 15 November 2026 with breaches planted on purpose. It has 13 crew, 24 flights and 62 duties.

2. Functional requirements

There are two roles. Write controls only render for the right role, and every API route checks the role again on the server, so hiding a button is never the only protection.

Role	Can do
Both roles	Sign in and out. Read the overview, crew, flights, duty roster, airports and fleet, validation runs, violations, rules, dashboard and activity log. Search and filter those lists.
SCHEDULER	Add, edit and delete crew members. Add and remove type ratings. Schedule and delete flights. Assign several crew to a flight in one transaction, and remove a duty.
AUDITOR	Run a validation over a date range. Review a violation by moving it from OPEN to ACKNOWLEDGED to RESOLVED, with a note.

Non-functional requirements from the guidelines: passwords are stored only as bcrypt hashes, credentials live in `.env.local` and are never committed, and all data validation happens in the database, with the interface showing the constraint that rejected a row.

3. Database design

3.1 ER diagram

This diagram replaces the one in Review 1. It corrects `violation_assignment`, which showed `resolution_notes` while the schema uses `contribution_value`, and adds the four review attributes and the reviewer relationship that came after Review 1.

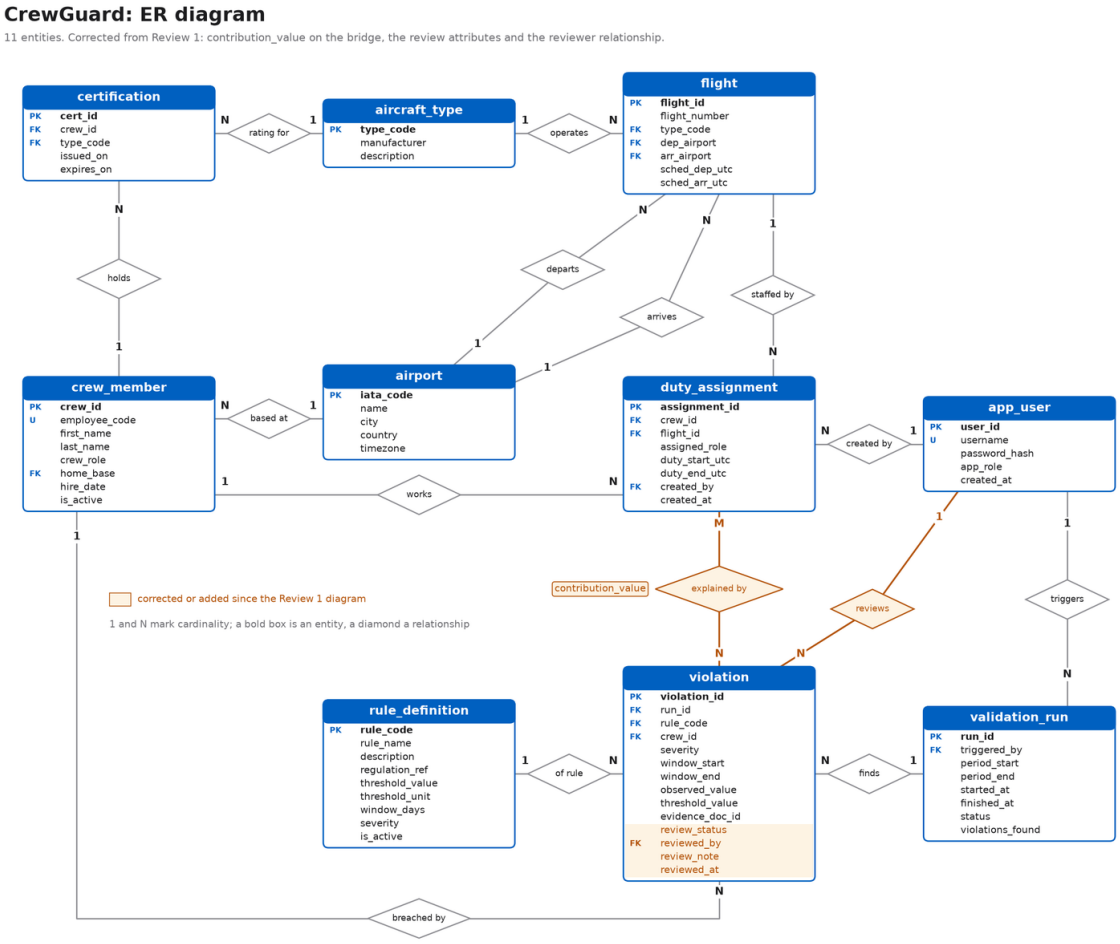


Figure 1: ER diagram. Corrected and added parts are in orange.

3.2 Relational schema

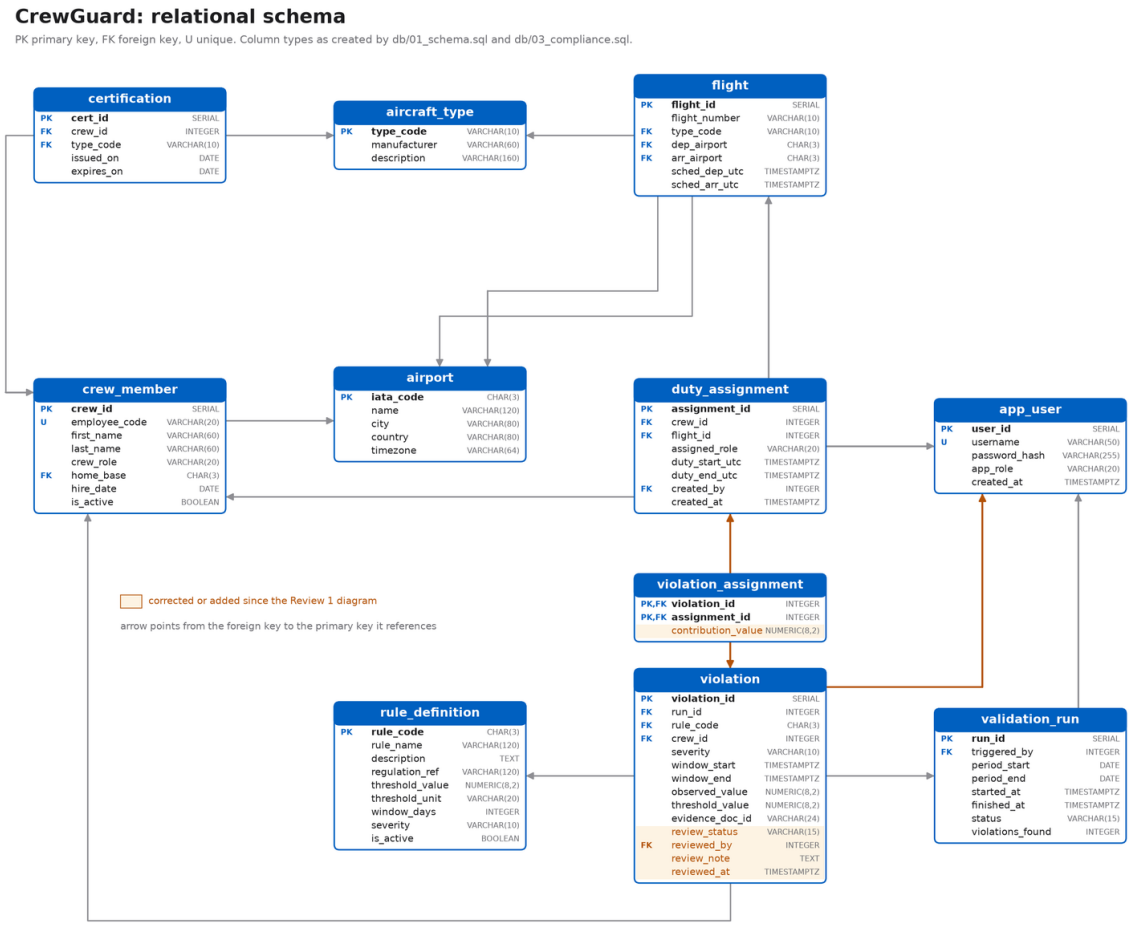


Figure 2: Relational schema, with every foreign key.

3.3 Normalisation

Each table is about one thing. Crew details are stored once in `crew_member` and duties only hold `crew_id`, airport and aircraft details live in their own tables, and a renewed rating is a new row in `certification`, not an update, so the history is kept. `violation_assignment` resolves the many-to-many relationship between violations and duties. Every non-key column depends on the whole key and nothing but the key, which puts the schema in third normal form.

There is one deliberate exception. `violation` copies `severity` and `threshold_value` from `rule_definition`. That is a snapshot, not redundancy: if a limit changes later, an old violation must still show the limit it was judged against.

3.4 Database choice and justification

Data	Store	Why
Airports, aircraft, users, crew, ratings, flights, duties, rules, runs, violations	PostgreSQL	Relational. Needs foreign keys, CHECK, UNIQUE and EXCLUDE constraints, transactions, and temporal SQL with window functions.
`violation_evidence`	MongoDB	The evidence has a different shape per rule (a rest gap, a 7-day breakdown, certificate details). A flexible document is cleaner than many nullable columns.

Data	Store	Why
audit_log	MongoDB	Append-only and write-heavy, with a different payload per action, and it is never joined.

PostgreSQL stores the MongoDB _id (24 hex characters) in violation.evidence_doc_id, and that column is the link between the two. PostgreSQL is the source of truth: the evidence is written after the validation transaction commits, and if MongoDB is down the violations still exist and the interface says the evidence is unavailable.

4. Data dictionary

Types, keys, defaults and constraints below are read from the database catalog. All times are stored in UTC.

airport

Airports, identified by IATA code.

Column	Type	Key	Not null	Default	Meaning
iata_code	char(3)	PK	yes		Three-letter IATA code, e.g. BLR
name	varchar(120)		yes		Full airport name
city	varchar(80)		yes		City served
country	varchar(80)		yes		Country
timezone	varchar(64)		yes		IANA time zone, e.g. Asia/Kolkata

```
chk_airport_iata_format: CHECK ((iata_code ~ '^[A-Z]{3}$'::text))
```

aircraft_type

Aircraft types crew can be rated on.

Column	Type	Key	Not null	Default	Meaning
type_code	varchar(10)	PK	yes		Type code, e.g. A320
manufacturer	varchar(60)		yes		Manufacturer
description	varchar(160)		no		Readable description

```
chk_aircraft_type_upper: CHECK (((type_code)::text = upper((type_code)::text)))
```

app_user

Login accounts.

Column	Type	Key	Not null	Default	Meaning
user_id	integer	PK	yes	auto	Account id
username	varchar(50)	U	yes		Login name
password_hash	varchar(255)		yes		bcrypt hash, never the plain password
app_role	varchar(20)		yes		SCHEDULER or AUDITOR
created_at	timestamptz		yes	now()	When the account was created

```
app_user_username_key: UNIQUE (username)
chk_app_role: CHECK (((app_role)::text = ANY ((ARRAY['SCHEDULER'::character varying, 'AUDITOR'::character
varying])::text[])))
```

crew_member

Pilots and cabin crew.

Column	Type	Key	Not null	Default	Meaning
crew_id	integer	PK	yes	auto	Crew id
employee_code	varchar(20)	U	yes		Airline employee code
first_name	varchar(60)		yes		First name
last_name	varchar(60)		yes		Last name
crew_role	varchar(20)		yes		CAPTAIN, FIRST_OFFICER or CABIN_CREW
home_base	char(3)	FK	yes		Airport they are based at
hire_date	date		yes		Date hired, not in the future
is_active	boolean		yes	true	Inactive crew cannot be rostered

```
fk_crew_home_base: FOREIGN KEY (home_base) REFERENCES airport(iata_code) ON UPDATE CASCADE ON DELETE RESTRICT
crew_member_employee_code_key: UNIQUE (employee_code)
chk_crew_hire_date: CHECK ((hire_date <= CURRENT_DATE))
chk_crew_role: CHECK (((crew_role)::text = ANY ((ARRAY['CAPTAIN'::character varying, 'FIRST_OFFICER'::character
varying, 'CABIN_CREW'::character varying])::text[])))
```

certification

Type ratings: which aircraft a crew member may operate and until when.

Column	Type	Key	Not null	Default	Meaning
cert_id	integer	PK	yes	auto	Certificate id
crew_id	integer	FK	yes		Who holds it
type_code	varchar(10)	FK	yes		Aircraft type it covers
issued_on	date		yes		First valid day
expires_on	date		yes		Last valid day, after issued_on

```
fk_cert_crew: FOREIGN KEY (crew_id) REFERENCES crew_member(crew_id) ON UPDATE CASCADE ON DELETE CASCADE
fk_cert_type: FOREIGN KEY (type_code) REFERENCES aircraft_type(type_code) ON UPDATE CASCADE ON DELETE RESTRICT
uk_cert_crew_type_issue: UNIQUE (crew_id, type_code, issued_on)
chk_cert_validity_order: CHECK ((expires_on > issued_on))
```

flight

Scheduled flights.

Column	Type	Key	Not null	Default	Meaning
flight_id	integer	PK	yes	auto	Flight id
flight_number	varchar(10)		yes		Number, e.g. CG101. Repeats daily, so unique with the departure time
type_code	varchar(10)	FK	yes		Aircraft type flown
dep_airport	char(3)	FK	yes		Departure airport
arr_airport	char(3)	FK	yes		Arrival airport, different from departure
sched_dep_utc	timestamptz		yes		Scheduled departure
sched_arr_utc	timestamptz		yes		Scheduled arrival, after departure

```
fk_flight_arr: FOREIGN KEY (arr_airport) REFERENCES airport(iata_code) ON UPDATE CASCADE ON DELETE RESTRICT
fk_flight_dep: FOREIGN KEY (dep_airport) REFERENCES airport(iata_code) ON UPDATE CASCADE ON DELETE RESTRICT
fk_flight_type: FOREIGN KEY (type_code) REFERENCES aircraft_type(type_code) ON UPDATE CASCADE ON DELETE RESTRICT
```

```
uk_flight_number_dep: UNIQUE (flight_number, sched_dep_utc)
chk_flight_distinct_endpoints: CHECK ((dep_airport <> arr_airport))
chk_flight_time_order: CHECK ((sched_arr_utc > sched_dep_utc))
```

duty_assignment

The roster: one crew member working one flight.

Column	Type	Key	Not null	Default	Meaning
assignment_id	integer	PK	yes	auto	Duty id
crew_id	integer	FK	yes		Crew member on duty
flight_id	integer	FK	yes		Flight worked
assigned_role	varchar(20)		yes		Role on this flight, must match the crew member's role
duty_start_utc	timestamptz		yes		Report time, at or before departure
duty_end_utc	timestamptz		yes		Release time, at or after arrival
created_by	integer	FK	no		Scheduler who rostered it
created_at	timestamptz		yes	now()	When it was rostered

```
fk_duty_created_by: FOREIGN KEY (created_by) REFERENCES app_user(user_id) ON UPDATE CASCADE ON DELETE SET NULL
fk_duty_crew: FOREIGN KEY (crew_id) REFERENCES crew_member(crew_id) ON UPDATE CASCADE ON DELETE CASCADE
fk_duty_flight: FOREIGN KEY (flight_id) REFERENCES flight(flight_id) ON UPDATE CASCADE ON DELETE CASCADE
uk_duty_crew_flight: UNIQUE (crew_id, flight_id)
chk_duty_assigned_role: CHECK (((assigned_role)::text = ANY ((ARRAY['CAPTAIN'::character varying,
'FIRST_OFFICER'::character varying, 'CABIN_CREW'::character varying])::text[])))
chk_duty_time_order: CHECK ((duty_end_utc > duty_start_utc))
excl_duty_no_overlap: EXCLUDE USING gist (crew_id WITH =, tstzrange(duty_start_utc, duty_end_utc) WITH &&)
```

rule_definition

The regulation catalogue. Limits live here, not in code.

Column	Type	Key	Not null	Default	Meaning
rule_code	char(3)	PK	yes		RST, FDP, WDH or CER
rule_name	varchar(120)		yes		Rule title
description	text		yes		What the rule requires
regulation_ref	varchar(120)		no		Regulation reference
threshold_value	numeric(8,2)		no		The limit, NULL for CER
threshold_unit	varchar(20)		no		Unit of the limit
window_days	integer		no		Rolling window length, only for WDH
severity	varchar(10)		yes		LOW, MEDIUM, HIGH or CRITICAL
is_active	boolean		yes	true	Inactive rules are skipped by the validator

```
chk_rule_severity: CHECK (((severity)::text = ANY ((ARRAY['LOW'::character varying, 'MEDIUM'::character varying,
'HIGH'::character varying, 'CRITICAL'::character varying])::text[])))
chk_rule_window_days: CHECK (((window_days IS NULL) OR (window_days > 0)))
```

validation_run

One execution of the validator.

Column	Type	Key	Not null	Default	Meaning
run_id	integer	PK	yes	auto	Run id
triggered_by	integer	FK	no		Auditor who ran it
period_start	date		yes		First day checked
period_end	date		yes		Last day checked

Column	Type	Key	Not null	Default	Meaning
started_at	timestamptz		yes	now()	When it started
finished_at	timestamptz		no		When it ended, set by trg_run_finish
status	varchar(15)		yes	'RUNNING'	RUNNING, COMPLETED, FAILED or STALE
violations_found	integer		yes	0	Kept correct by trg_violation_count

```
fk_run_user: FOREIGN KEY (triggered_by) REFERENCES app_user(user_id) ON UPDATE CASCADE ON DELETE SET NULL
chk_run_period_order: CHECK ((period_end >= period_start))
chk_run_status: CHECK (((status)::text = ANY ((ARRAY['RUNNING'::character varying, 'COMPLETED'::character varying,
'FAILED'::character varying, 'STALE'::character varying])::text[])))
chk_run_violation_count: CHECK ((violations_found >= 0))
```

violation

One detected breach of one rule by one crew member in one run.

Column	Type	Key	Not null	Default	Meaning
violation_id	integer	PK	yes	auto	Violation id
run_id	integer	FK	yes		Run that found it
rule_code	char(3)	FK	yes		Rule breached
crew_id	integer	FK	yes		Crew member involved
severity	varchar(10)		yes		Copied from the rule when detected
window_start	timestamptz		no		Start of the breach window
window_end	timestamptz		no		End of the breach window
observed_value	numeric(8,2)		no		The measured hours, NULL for CER
threshold_value	numeric(8,2)		no		The limit when detected, NULL for CER
evidence_doc_id	varchar(24)		no		MongoDB _id of the evidence document
review_status	varchar(15)		yes	'OPEN'	OPEN, ACKNOWLEDGED or RESOLVED
reviewed_by	integer	FK	no		Auditor who last reviewed it
review_note	text		no		Required before it can be RESOLVED
reviewed_at	timestamptz		no		When the status last changed, set by trg_violation_review

```
fk_violation_crew: FOREIGN KEY (crew_id) REFERENCES crew_member(crew_id) ON UPDATE CASCADE ON DELETE CASCADE
fk_violation_reviewer: FOREIGN KEY (reviewed_by) REFERENCES app_user(user_id) ON UPDATE CASCADE ON DELETE SET NULL
fk_violation_rule: FOREIGN KEY (rule_code) REFERENCES rule_definition(rule_code) ON UPDATE CASCADE ON DELETE RESTRICT
fk_violation_run: FOREIGN KEY (run_id) REFERENCES validation_run(run_id) ON UPDATE CASCADE ON DELETE CASCADE
chk_violation_resolution_note: CHECK ((((review_status)::text <> 'RESOLVED'::text) OR (review_note IS NOT NULL)))
chk_violation_review_status: CHECK (((review_status)::text = ANY ((ARRAY['OPEN'::character varying,
'ACKNOWLEDGED'::character varying, 'RESOLVED'::character varying])::text[])))
chk_violation_severity: CHECK (((severity)::text = ANY ((ARRAY['LOW'::character varying, 'MEDIUM'::character varying,
'HIGH'::character varying, 'CRITICAL'::character varying])::text[])))
chk_violation_window_order: CHECK (((window_end IS NULL) OR (window_start IS NULL) OR (window_end >= window_start)))
```

violation_assignment

The explanation: which duties caused which violation (M:N).

Column	Type	Key	Not null	Default	Meaning
violation_id	integer	PK, FK	yes		The violation
assignment_id	integer	PK, FK	yes		A duty that caused it
contribution_value	numeric(8,2)		no		Hours this duty contributed

```
fk_va_assignment: FOREIGN KEY (assignment_id) REFERENCES duty_assignment(assignment_id) ON UPDATE CASCADE ON DELETE
CASCADE
fk_va_violation: FOREIGN KEY (violation_id) REFERENCES violation(violation_id) ON UPDATE CASCADE ON DELETE CASCADE
```

5. SQL implementation

File	Contents
db/01_schema.sql	DROP, CREATE for the 11 tables, keys, constraints, indexes, table comments
db/02_roster.sql	Four functions, v_duty_roster, v_crew_summary, trg_validate_duty
db/03_compliance.sql	ALTER for the review workflow, three triggers, v_violation_report, sp_run_validation, full-text search
db/04_seed.sql	The sample data. No runs or violations are seeded, the validator creates them
db/queries.sql	The important queries, runnable in psql
db/reset.mjs	Rebuilds the database from the numbered files and empties the MongoDB collections

5.1 DDL and constraints

Every rule that can be expressed in the database is. Section 4 lists all the constraints. The one that a plain UNIQUE or CHECK cannot express is the no-overlap rule: it compares time ranges, so it needs an exclusion constraint backed by a GiST index.

```
ALTER TABLE duty_assignment
    ADD CONSTRAINT excl_duty_no_overlap
    EXCLUDE USING GIST (
        crew_id WITH =,
        tstzrange(duty_start_utc, duty_end_utc) WITH &&
    );
```

The foreign keys use all three delete behaviours on purpose. An airport that is someone's home base cannot be deleted (RESTRICT), deleting a crew member removes their ratings and duties (CASCADE), and removing a scheduler's account keeps the duties but forgets the author (SET NULL).

5.2 ALTER: the review workflow

Review columns were added with ALTER because the workflow came after Review 1, so 01_schema.sql still matches that schema. The second CHECK means a violation cannot be closed without saying how it was resolved.

```
ALTER TABLE violation
    ADD COLUMN review_status VARCHAR(15) NOT NULL DEFAULT 'OPEN',
    ADD COLUMN reviewed_by   INTEGER,
    ADD COLUMN review_note   TEXT,
    ADD COLUMN reviewed_at   TIMESTAMPTZ,
    ADD CONSTRAINT chk_violation_review_status
        CHECK (review_status IN ('OPEN', 'ACKNOWLEDGED', 'RESOLVED')),
    -- if the auditor's account is removed, keep the review but forget who did it
    ADD CONSTRAINT fk_violation_reviewer FOREIGN KEY (reviewed_by) REFERENCES app_user(user_id)
        ON UPDATE CASCADE ON DELETE SET NULL,
-- a violation cannot be closed without saying how it was resolved
    ADD CONSTRAINT chk_violation_resolution_note
        CHECK (review_status <> 'RESOLVED' OR review_note IS NOT NULL);
```

5.3 Views

View	What it does
v_duty_roster	Every duty joined to its crew member and flight, with the duty hours. The roster page and the explanation query read this.

View	What it does
v_crew_summary	One row per crew member, using LEFT JOINs to two GROUP BY sub-queries so counts do not multiply and crew with no duties still appear.
v_violation_report	One readable row per violation, with a LEFT JOIN for the reviewer and a correlated sub-query for the duty count.

```
CREATE OR REPLACE VIEW v_violation_report AS
SELECT v.violation_id,
       v.run_id,
       vr.period_start,
       vr.period_end,
       v.rule_code,
       r.rule_name,
       v.severity,
       v.crew_id,
       cm.employee_code,
       cm.first_name || ' ' || cm.last_name AS crew_name,
       cm.crew_role,
       v.window_start,
       v.window_end,
       v.observed_value,
       v.threshold_value,
       v.evidence_doc_id,
       v.review_status,
       u.username AS reviewer,
       v.review_note,
       v.reviewed_at,
       (SELECT COUNT(*)::int
        FROM violation_assignment va
        WHERE va.violation_id = v.violation_id) AS duty_count
FROM violation v
JOIN validation_run  vr ON vr.run_id   = v.run_id
JOIN rule_definition r  ON r.rule_code = v.rule_code
JOIN crew_member     cm ON cm.crew_id  = v.crew_id
LEFT JOIN app_user   u  ON u.user_id   = v.reviewed_by;
```

5.4 Functions

Function	Kind	What it does
fn_duty_hours(start, end)	scalar, IMMUTABLE	Hours between two timestamps, to 2 decimals. The validator uses it everywhere.
fn_crew_duty_hours(crew, from, to)	aggregate, STABLE	Total duty hours of one crew member in a time range.
fn_crew_roster(crew)	table-valued	One crew member's duties, usable like a table.
fn_cert_status(expires_on, on_date)	scalar, STABLE	EXPIRED, EXPIRING (within 60 days) or VALID.

5.5 Triggers

Trigger	When	Why it is a trigger
trg_validate_duty	BEFORE INSERT OR UPDATE on duty_assignment	Rejects inactive crew, a wrong role, or a duty not covering its flight. These need other tables, and a CHECK can only see the row being written.
trg_violation_count	AFTER INSERT OR DELETE on violation	Keeps violations_found correct however a violation is added or removed, including by cascade.
trg_run_finish	BEFORE UPDATE on validation_run	Stamps finished_at when a run is closed.
trg_violation_review	BEFORE UPDATE OF review_status on violation	Stamps reviewed_at when the status actually changes, not when only the note is edited.

`trg_run_finish` uses `clock_timestamp()` and not `NOW()`. `NOW()` is frozen at the start of the transaction, so a run closed in the same transaction that opened it would always show a duration of zero.

```
CREATE OR REPLACE FUNCTION trg_fn_run_finish()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.status IN ('COMPLETED', 'FAILED') AND NEW.finished_at IS NULL THEN
        NEW.finished_at := clock_timestamp();
    END IF;
    RETURN NEW;
END;
$$;
```

5.6 Stored procedure: the validator

`sp_run_validation` is the core of the project. It opens a run, checks each active rule, records each breach in `violation`, links it to the duties that caused it in `violation_assignment`, and closes the run. It reads thresholds and severities from `rule_definition`, so a limit can change without touching this code, and a test proves that. It does not commit: the caller's transaction makes the whole run atomic.

Rule	Detection	Duties linked
RST	`LAG()` over all of a crew member's duties in time order gives the previous duty. Report it when the gap is under the limit and the current duty is in the period.	The previous and the current duty
FDP	Duty length over the limit, for duties in the period.	That duty
WDH	Rolling `SUM() OVER (... RANGE BETWEEN 7 days PRECEDING AND CURRENT ROW)`. `RANGE` because the window is 7 days of time, not 7 rows. `DISTINCT ON` keeps each crew member's single worst window, so one overworked week is one violation and not one per duty.	Every duty in that window
CER	`NOT EXISTS` a certificate for this crew and aircraft type covering the duty date. Renewals are separate rows, so an expired certificate is fine if a newer one covers the date. Cabin crew are excluded.	That duty

Three details matter. RST and WDH compute over all duties and only report on duties inside the period, because the previous duty or the window can begin before the period starts. Period bounds and duty dates are converted with `AT TIME ZONE 'UTC'`, so the result does not depend on the session's time zone, and a test proves that as well. And each crew member gets at most one WDH violation per run.

```
CREATE OR REPLACE PROCEDURE sp_run_validation(p_user_id INTEGER,
                                              p_start   DATE,
                                              p_end     DATE,
                                              INOUT p_run_id INTEGER DEFAULT NULL)
LANGUAGE plpgsql
AS $$
DECLARE
    -- AT TIME ZONE 'UTC': the result must not depend on the session time zone
    v_from TIMESTAMPTZ := p_start::timestamp AT TIME ZONE 'UTC';
    v_to   TIMESTAMPTZ := (p_end + 1)::timestamp AT TIME ZONE 'UTC';
    v_rule rule_definition%ROWTYPE;
    v_id   INTEGER;
    r      RECORD;
BEGIN
    INSERT INTO validation_run (triggered_by, period_start, period_end)
    VALUES (p_user_id, p_start, p_end)
    RETURNING run_id INTO p_run_id;

    -- ---------- RST: minimum rest between two consecutive duties ----------
    SELECT * INTO v_rule FROM rule_definition WHERE rule_code = 'RST';
    IF v_rule.is_active THEN
        FOR r IN
```

```
        -- LAG reads the same crew member's previous duty, in time order
        SELECT d.*, fn_duty_hours(d.prev_end, d.duty_start_utc) AS rest_hours
        FROM (
            SELECT da.assignment_id, da.crew_id, da.duty_start_utc, da.duty_end_utc,
                   LAG(da.assignment_id)  OVER w AS prev_id,
                   LAG(da.duty_start_utc) OVER w AS prev_start,
                   LAG(da.duty_end_utc)   OVER w AS prev_end
            FROM duty_assignment da
            WINDOW w AS (PARTITION BY da.crew_id ORDER BY da.duty_start_utc)
        ) d
        WHERE d.prev_id IS NOT NULL
          AND d.duty_start_utc >= v_from AND d.duty_start_utc < v_to
          AND fn_duty_hours(d.prev_end, d.duty_start_utc) < v_rule.threshold_value
        ORDER BY d.crew_id, d.duty_start_utc
    LOOP
        INSERT INTO violation (run_id, rule_code, crew_id, severity, window_start, window_end,
                               observed_value, threshold_value)
        VALUES (p_run_id, v_rule.rule_code, r.crew_id, v_rule.severity, r.prev_end, r.duty_start_utc,
                r.rest_hours, v_rule.threshold_value)
        RETURNING violation_id INTO v_id;

        -- explanation: the two duties the short rest sits between
        INSERT INTO violation_assignment (violation_id, assignment_id, contribution_value)
        VALUES (v_id, r.prev_id,       fn_duty_hours(r.prev_start, r.prev_end)),
               (v_id, r.assignment_id, fn_duty_hours(r.duty_start_utc, r.duty_end_utc));
    END LOOP;
END IF;

-- ---------- FDP: maximum length of a single duty ----------
SELECT * INTO v_rule FROM rule_definition WHERE rule_code = 'FDP';
IF v_rule.is_active THEN
    FOR r IN
        SELECT da.assignment_id, da.crew_id, da.duty_start_utc, da.duty_end_utc,
               fn_duty_hours(da.duty_start_utc, da.duty_end_utc) AS hours
        FROM duty_assignment da
        WHERE da.duty_start_utc >= v_from AND da.duty_start_utc < v_to
          AND fn_duty_hours(da.duty_start_utc, da.duty_end_utc) > v_rule.threshold_value
        ORDER BY da.crew_id, da.duty_start_utc
    LOOP
        INSERT INTO violation (run_id, rule_code, crew_id, severity, window_start, window_end,
                               observed_value, threshold_value)
        VALUES (p_run_id, v_rule.rule_code, r.crew_id, v_rule.severity, r.duty_start_utc, r.duty_end_utc,
                r.hours, v_rule.threshold_value)
        RETURNING violation_id INTO v_id;

        INSERT INTO violation_assignment (violation_id, assignment_id, contribution_value)
        VALUES (v_id, r.assignment_id, r.hours);
    END LOOP;
END IF;

-- ---------- WDH: maximum duty hours in any rolling window ----------
-- The window function gives every duty the total of that crew member's duties
-- in the window ending at it. DISTINCT ON then keeps each crew member's single
-- worst window (the earliest if tied), so one overworked week is ONE violation
-- per crew per run, not one for every duty inside that week.
SELECT * INTO v_rule FROM rule_definition WHERE rule_code = 'WDH';
IF v_rule.is_active THEN
    FOR r IN
        SELECT DISTINCT ON (d.crew_id) d.crew_id, d.duty_start_utc, d.duty_end_utc, d.rolling_hours
        FROM (
            SELECT da.crew_id, da.duty_start_utc, da.duty_end_utc,
                   SUM(fn_duty_hours(da.duty_start_utc, da.duty_end_utc)) OVER (
                       PARTITION BY da.crew_id
                       ORDER BY da.duty_start_utc
                       RANGE BETWEEN make_interval(days => v_rule.window_days) PRECEDING AND CURRENT ROW
                   ) AS rolling_hours
            FROM duty_assignment da
        ) d
        WHERE d.duty_start_utc >= v_from AND d.duty_start_utc < v_to
          AND d.rolling_hours > v_rule.threshold_value
        ORDER BY d.crew_id, d.rolling_hours DESC, d.duty_start_utc
    LOOP
        INSERT INTO violation (run_id, rule_code, crew_id, severity, window_start, window_end,
                               observed_value, threshold_value)
        VALUES (p_run_id, v_rule.rule_code, r.crew_id, v_rule.severity,
                r.duty_start_utc - make_interval(days => v_rule.window_days), r.duty_end_utc,
                r.rolling_hours, v_rule.threshold_value)
        RETURNING violation_id INTO v_id;

        -- explanation: every duty inside the window, each with its hours
        INSERT INTO violation_assignment (violation_id, assignment_id, contribution_value)
```

```
            SELECT v_id, da.assignment_id, fn_duty_hours(da.duty_start_utc, da.duty_end_utc)
            FROM duty_assignment da
            WHERE da.crew_id = r.crew_id
              AND da.duty_start_utc >= r.duty_start_utc - make_interval(days => v_rule.window_days)
              AND da.duty_start_utc <= r.duty_start_utc;
        END LOOP;
    END IF;

    -- ---------- CER: pilot must hold a valid type rating ----------
    -- Anti-join: no certificate covers this aircraft type on the duty date.
    -- A renewal is a separate row, so an expired certificate does not trigger this
    -- as long as a newer one covers the date. Cabin crew are exempt.
    SELECT * INTO v_rule FROM rule_definition WHERE rule_code = 'CER';
    IF v_rule.is_active THEN
        FOR r IN
            SELECT da.assignment_id, da.crew_id, da.duty_start_utc, da.duty_end_utc,
                   fn_duty_hours(da.duty_start_utc, da.duty_end_utc) AS hours
            FROM duty_assignment da
            JOIN flight f ON f.flight_id = da.flight_id
            WHERE da.duty_start_utc >= v_from AND da.duty_start_utc < v_to
              AND da.assigned_role <> 'CABIN_CREW'
              AND NOT EXISTS (
                  SELECT 1 FROM certification c
                  WHERE c.crew_id = da.crew_id
                    AND c.type_code = f.type_code
                    AND (da.duty_start_utc AT TIME ZONE 'UTC')::date BETWEEN c.issued_on AND c.expires_on)
            ORDER BY da.crew_id, da.duty_start_utc
        LOOP
            -- there is no number to compare here, so observed and threshold stay NULL
            INSERT INTO violation (run_id, rule_code, crew_id, severity, window_start, window_end)
            VALUES (p_run_id, v_rule.rule_code, r.crew_id, v_rule.severity, r.duty_start_utc, r.duty_end_utc)
            RETURNING violation_id INTO v_id;

            INSERT INTO violation_assignment (violation_id, assignment_id, contribution_value)
            VALUES (v_id, r.assignment_id, r.hours);
        END LOOP;
    END IF;

    -- trg_run_finish stamps finished_at; trg_violation_count already kept the count
    UPDATE validation_run SET status = 'COMPLETED' WHERE run_id = p_run_id;
END;
$$;
```

5.7 Indexes

PostgreSQL indexes primary keys and unique columns automatically, but not foreign keys, so the join and filter columns are indexed by hand.

Index	On	Type	Used by
idx_duty_crew_start	duty_assignment (crew_id, duty_start_utc)	B-tree, composite	Crew roster, and the validator's per-crew time order
idx_duty_flight	duty_assignment (flight_id)	B-tree	Flight detail
idx_flight_dep_time	flight (sched_dep_utc)	B-tree	Flight date filter
idx_cert_crew_type	certification (crew_id, type_code, expires_on)	B-tree, composite	CER rule and rating checks
idx_crew_home_base	crew_member (home_base)	B-tree	Base filter
idx_violation_run	violation (run_id)	B-tree	Run detail
idx_violation_crew	violation (crew_id)	B-tree	Crew search on violations
idx_violation_rule	violation (rule_code)	B-tree	Rule filter, dashboard grouping
idx_va_assignment	violation_assignment (assignment_id)	B-tree	Reverse lookup from a duty
excl_duty_no_overlap	duty_assignment (crew_id, time range)	GiST	The no-overlap constraint
idx_rule_fts	rule_definition (expression)	GIN	Full-text search on the rules

5.8 Transactions

There are two all-or-nothing writes. Assigning several crew to a flight is one transaction: if any insert is rejected, `ROLLBACK` undoes the ones that already succeeded, so a flight is never left half-staffed. Running a validation is the other: the run, its violations and their links either all commit or are all gone.

```
export function runValidation(userId: number, periodStart: unknown, periodEnd: unknown) {
  return withTransaction(async (client) => {
    const { rows } = await client.query<{ p_run_id: number }>(
      "CALL sp_run_validation($1, $2, $3, NULL)",
      [userId, periodStart, periodEnd],
    );
    return rows[0].p_run_id;
  });
}
```

5.9 Full-text search

The rule catalogue is searchable with a GIN index. `websearch_to_tsquery` accepts what people type into a search box and never raises a syntax error, `ts_rank` orders the results, and `ts_headline` marks the matched words. Stemming means a search for "certificates" finds "certification". The search expression must match the indexed expression exactly, or the index is not used, so it is defined once in the code.

```
CREATE INDEX idx_rule_fts ON rule_definition
    USING GIN (to_tsvector('english', rule_name || ' ' || description || ' ' ||
COALESCE(regulation_ref, '')));
```

6. NoSQL implementation

6.1 Collections

Collection	One document per	Indexes
`violation_evidence`	violation	`{violation_id: 1}` unique, `{run_id: 1}`, `{rule_code: 1, severity: 1}`
`audit_log`	user action	`{ts: -1}`, `{action: 1, ts: -1}`

The indexes are created when the app first connects. `createIndexes` does nothing if an index already exists, so this is safe on every start.

6.2 Sample documents

These are real documents from a test run against MongoDB. The common fields are the same for every rule and `evidence` changes shape, which is exactly why this data is in a document store. Four evidence shapes in one relational table would mean a long list of mostly empty columns. The `_id` of each document is what `violation.evidence_doc_id` stores.

RST: a rest gap

```
{
  "_id": "ObjectId(6ae8c35701b127f991804e38)",
  "violation_id": 1,
  "run_id": 1,
  "rule_code": "RST",
  "severity": "HIGH",
  "created_at": "2026-11-02T17:07:35.223Z",
  "crew": { "crew_id": 1, "employee_code": "EMP1001", "name": "Arjun Menon", "role": "CAPTAIN" },
```

```
  "evidence": {
    "previous_duty": { "flight_number": "CG101", "end": "2026-11-02T04:15:00Z" },
    "next_duty": { "flight_number": "CG102", "start": "2026-11-02T04:30:00Z" },
    "rest_hours": 0.25,
    "required_hours": 10
  }
}
```


FDP: one long duty


```
{
  "_id": "ObjectId(6ae8c35701b127f991804e3f)",
  "violation_id": 8,
  "run_id": 1,
  "rule_code": "FDP",
  "severity": "CRITICAL",
  "created_at": "2026-11-02T17:07:35.223Z",
  "crew": { "crew_id": 7, "employee_code": "EMP1007", "name": "Aditya Reddy", "role": "CAPTAIN" },
  "evidence": {
    "flight_number": "CG131",
    "sector": "BOM → LHR",
    "duty_start": "2026-11-09T01:00:00Z",
    "duty_end": "2026-11-09T15:00:00Z",
    "duty_hours": 14,
    "limit_hours": 13
  }
}
```


WDH: a 7-day breakdown


```
{
  "_id": "ObjectId(6ae8c35701b127f991804e42)",
  "violation_id": 11,
  "run_id": 1,
  "rule_code": "WDH",
  "severity": "MEDIUM",
  "created_at": "2026-11-02T17:07:35.223Z",
  "crew": { "crew_id": 9, "employee_code": "EMP1009", "name": "Farhan Ali", "role": "CAPTAIN" },
  "evidence": {
    "window_start": "2026-10-31T00:00:00Z",
    "window_end": "2026-11-07T11:00:00Z",
    "total_hours": 66,
    "limit_hours": 60,
    "duties": [
      { "flight_number": "CG111", "start": "2026-11-02T00:00:00Z", "hours": 11 },
      { "flight_number": "CG112", "start": "2026-11-03T00:00:00Z", "hours": 11 },
      { "flight_number": "CG111", "start": "2026-11-04T00:00:00Z", "hours": 11 },
      { "flight_number": "CG112", "start": "2026-11-05T00:00:00Z", "hours": 11 },
      { "flight_number": "CG111", "start": "2026-11-06T00:00:00Z", "hours": 11 },
      { "flight_number": "CG112", "start": "2026-11-07T00:00:00Z", "hours": 11 }
    ]
  }
}
```


CER: no rating held


```
{
  "_id": "ObjectId(6ae8c35701b127f991804e44)",
  "violation_id": 13,
  "run_id": 1,
  "rule_code": "CER",
  "severity": "CRITICAL",
  "created_at": "2026-11-02T17:07:35.223Z",
  "crew": { "crew_id": 3, "employee_code": "EMP1003", "name": "Rohan Kumar", "role": "FIRST_OFFICER"
},
  "evidence": {
    "flight_number": "CG201",
    "aircraft_type": "B738",
    "duty_date": "2026-11-03",
    "reason": "NO_RATING",
    "ratings_held": [
      { "type_code": "A320", "issued_on": "2024-05-20", "expires_on": "2027-05-19" },
      { "type_code": "A321", "issued_on": "2025-06-18", "expires_on": "2028-06-17" }
    ]
```

```
  }
}
```

CER: an expired rating

```
{
  "_id": "ObjectId(6ae8c35701b127f991804e46)",
  "violation_id": 15,
  "run_id": 1,
  "rule_code": "CER",
  "severity": "CRITICAL",
  "created_at": "2026-11-02T17:07:35.223Z",
  "crew": { "crew_id": 4, "employee_code": "EMP1004", "name": "Sneha Patil", "role": "FIRST_OFFICER"
},
  "evidence": {
    "flight_number": "CG211",
    "aircraft_type": "B738",
    "duty_date": "2026-11-09",
    "reason": "EXPIRED",
    "ratings_held": [
      { "type_code": "B738", "issued_on": "2023-08-12", "expires_on": "2026-08-11" }
    ]
  }
}
```

audit_log

The actions are LOGIN, CREW_CREATE, CREW_UPDATE, CREW_DELETE, CERT_ADD, CERT_DELETE, FLIGHT_CREATE, FLIGHT_DELETE, DUTY_ASSIGN, DUTY_DELETE, VALIDATION_RUN and VIOLATION_REVIEW. Two real entries:

```
{
  "_id": "ObjectId(6ae8c35701b127f991804e48)",
  "ts": "2026-11-02T17:07:35.234Z",
  "user": { "id": 2, "username": "audit_shriram", "role": "AUDITOR" },
  "action": "VALIDATION_RUN",
  "entity": { "type": "run", "id": 1 },
  "details": { "period_start": "2026-11-02", "period_end": "2026-11-15" }
}

{
  "_id": "ObjectId(6ae8c35701b127f991804e49)",
  "ts": "2026-11-02T17:07:35.264Z",
  "user": { "id": 2, "username": "audit_shriram", "role": "AUDITOR" },
  "action": "VIOLATION_REVIEW",
  "entity": { "type": "violation", "id": 1 },
  "details": { "review_status": "ACKNOWLEDGED" }
}
```

6.3 Aggregation pipelines

A pipeline passes documents through stages in order. These feed the dashboard, next to one panel computed in SQL so both databases can be compared.

```
// violations per rule
      evidence<Count>([
        { $group: { _id: "$rule_code", count: { $sum: 1 } } },
        { $sort: { count: -1, _id: 1 } },
      ]),
      // violations per severity
      evidence<Count>([
        { $group: { _id: "$severity", count: { $sum: 1 } } },
        { $sort: { count: -1, _id: 1 } },
      ]),
      // the five crew members with the most violations
      evidence<CrewCount>([
        { $group: { _id: "$crew.crew_id", name: { $first: "$crew.name" }, count: { $sum: 1 } } },
        { $sort: { count: -1, name: 1 } },
        { $limit: 5 },
      ]),
      // average total hours across the WDH breaches: a field only WDH documents have
      evidence<{ avg_hours: number; count: number }>([
        { $match: { rule_code: "WDH" } },
        { $group: { _id: null, avg_hours: { $avg: "$evidence.total_hours" }, count: { $sum: 1 } } },
```

```
      ]),
      // violations per run, latest ten
      evidence<Count>([
        { $group: { _id: "$run_id", count: { $sum: 1 } } },
        { $sort: { _id: -1 } },
        { $limit: 10 },
      ]),
      // logged actions per day, latest fourteen days
      run<Count>("audit_log", [
        { $group: { _id: { $dateToString: { format: "%Y-%m-%d", date: "$ts" } }, count: { $sum: 1 } } },
        { $sort: { _id: -1 } },
        { $limit: 14 },
      ]),
```

6.4 Writing evidence, and what happens when MongoDB is down

Evidence is written after the validation transaction commits: read the run's violations and linked duties from PostgreSQL, build one document per violation, `insertMany`, then store each `_id` in `violation.evidence_doc_id`.

A MongoDB failure never fails a request. The connection gives up after 2 seconds, and callers catch the error. If MongoDB is down, runs and reviews still work, roster writes still save, the evidence panel says the evidence is unavailable, and the dashboard keeps its PostgreSQL panel.

7. Important queries

These come from `db/queries.sql` and show the real output from psql. Part 2's writes run inside `BEGIN ... ROLLBACK`, so running the file leaves the data unchanged.

Q1 View: the operational roster (3-table JOIN inside v_duty_roster)

A view hides a 3-table join.

```
-- Q1  View: the operational roster (3-table JOIN inside v_duty_roster)

SELECT crew_name, assigned_role, flight_number, dep_airport, arr_airport, duty_start_utc, duty_hours

FROM v_duty_roster

WHERE duty_start_utc::date = '2026-11-02'

ORDER BY duty_start_utc;

  crew_name   | assigned_role | flight_number | dep_airport | arr_airport |     duty_start_utc     | duty_hours
--------------+---------------+---------------+-------------+-------------+------------------------+------------
 Vikram Joshi | CABIN_CREW    | CG101         | BLR         | DEL         | 2026-11-02 00:00:00+00 |       4.25
 Rohan Kumar  | FIRST_OFFICER | CG101         | BLR         | DEL         | 2026-11-02 00:00:00+00 |       4.25
 Arjun Menon  | CAPTAIN       | CG101         | BLR         | DEL         | 2026-11-02 00:00:00+00 |       4.25
 Karan Mehta  | FIRST_OFFICER | CG111         | DEL         | LHR         | 2026-11-02 00:00:00+00 |      11.00
 Farhan Ali   | CAPTAIN       | CG111         | DEL         | LHR         | 2026-11-02 00:00:00+00 |      11.00
 Nisha Shah   | CABIN_CREW    | CG111         | DEL         | LHR         | 2026-11-02 00:00:00+00 |      11.00
 Vikram Joshi | CABIN_CREW    | CG102         | DEL         | BLR         | 2026-11-02 04:30:00+00 |       4.25
 Rohan Kumar  | FIRST_OFFICER | CG102         | DEL         | BLR         | 2026-11-02 04:30:00+00 |       4.25
 Arjun Menon  | CAPTAIN       | CG102         | DEL         | BLR         | 2026-11-02 04:30:00+00 |       4.25
(9 rows)
```

Q3 LEFT JOIN + GROUP BY + FILTER: crew count per flight, unstaffed flights included

LEFT JOIN keeps unstaffed flights, FILTER counts pilots separately.

```
-- Q3  LEFT JOIN + GROUP BY + FILTER: crew count per flight, unstaffed flights included

SELECT f.flight_number, f.dep_airport || ' → ' || f.arr_airport AS route,

       COUNT(da.assignment_id)                                          AS crew,

       COUNT(da.assignment_id) FILTER (WHERE da.assigned_role <> 'CABIN_CREW') AS pilots

FROM flight f

LEFT JOIN duty_assignment da ON da.flight_id = f.flight_id
```

```
GROUP BY f.flight_id

ORDER BY f.sched_dep_utc;

 flight_number |   route   | crew | pilots
---------------+-----------+------+--------
 CG111         | DEL → LHR |    3 |      2
 CG101         | BLR → DEL |    3 |      2
 CG102         | DEL → BLR |    3 |      2
 CG112         | LHR → DEL |    3 |      2
 CG201         | BLR → BOM |    3 |      2
 CG202         | BOM → BLR |    3 |      2
 CG111         | DEL → LHR |    3 |      2
 CG301         | BLR → MAA |    2 |      1
 CG302         | MAA → BLR |    2 |      1
 CG401         | BOM → DXB |    2 |      2
 CG112         | LHR → DEL |    3 |      2
 CG402         | DXB → BOM |    2 |      2
 CG111         | DEL → LHR |    3 |      2
 CG501         | BLR → HYD |    2 |      1
 CG502         | HYD → BLR |    2 |      1
 CG112         | LHR → DEL |    3 |      2
... (9 more lines)
```

Q4 GROUP BY + HAVING: crew with more than 30 rostered hours

HAVING filters after grouping.

```
-- Q4  GROUP BY + HAVING: crew with more than 30 rostered hours

SELECT crew_id, COUNT(*) AS duties, SUM(fn_duty_hours(duty_start_utc, duty_end_utc)) AS hours

FROM duty_assignment

GROUP BY crew_id

HAVING SUM(fn_duty_hours(duty_start_utc, duty_end_utc)) > 30

ORDER BY hours DESC;

 crew_id | duties | hours
---------+--------+-------
       6 |      8 | 73.34
      11 |      6 | 66.00
       9 |      6 | 66.00
      10 |      4 | 35.00
       7 |      4 | 35.00
(5 rows)
```

Q6 NOT EXISTS sub-query: ratings expired/expiring and NOT yet renewed

NOT EXISTS drops certificates that have since been renewed.

```
-- Q6  NOT EXISTS sub-query: ratings expired/expiring and NOT yet renewed

SELECT cm.first_name || ' ' || cm.last_name AS crew_name, c.type_code, c.expires_on,

       fn_cert_status(c.expires_on) AS status

FROM certification c

JOIN crew_member cm ON cm.crew_id = c.crew_id

WHERE fn_cert_status(c.expires_on) <> 'VALID'

  AND NOT EXISTS (SELECT 1 FROM certification newer

                  WHERE newer.crew_id = c.crew_id AND newer.type_code = c.type_code

                    AND newer.issued_on > c.issued_on)

ORDER BY c.expires_on;

  crew_name  | type_code | expires_on |  status
-------------+-----------+------------+----------
 Farhan Ali  | A320      | 2025-01-01 | EXPIRED
 Sneha Patil | B738      | 2026-08-11 | EXPIRED
 Arjun Menon | A321      | 2026-12-20 | EXPIRING
 Karan Mehta | B788      | 2026-12-31 | EXPIRING
(4 rows)
```

Q7 Window function: mark the newest certificate per aircraft type

A window function marks the newest certificate per type.

```
-- Q7  Window function: mark the newest certificate per aircraft type

SELECT type_code, issued_on, expires_on,

       ROW_NUMBER() OVER (PARTITION BY type_code ORDER BY issued_on DESC) = 1 AS is_current

FROM certification

WHERE crew_id = 2;

 type_code | issued_on  | expires_on | is_current
-----------+------------+------------+------------
 A320      | 2026-10-20 | 2029-10-19 | t
 A320      | 2023-11-01 | 2026-10-31 | f
 B738      | 2025-01-05 | 2028-01-04 | t
(3 rows)
```

Q8 Table-valued function: one crew member's roster

A table-valued function used like a table.

```
-- Q8  Table-valued function: one crew member's roster

SELECT * FROM fn_crew_roster(1);

 assignment_id | flight_id | flight_number | type_code |  sector   | assigned_role |     duty_start_utc     |
duty_end_utc      | duty_hours
---------------+-----------+---------------+-----------+-----------+---------------+------------------------+--------
---------------+------------
             1 |         1 | CG101         | A320      | BLR → DEL | CAPTAIN       | 2026-11-02 00:00:00+00 | 2026-
11-02 04:15:00+00 |       4.25
             4 |         2 | CG102         | A320      | DEL → BLR | CAPTAIN       | 2026-11-02 04:30:00+00 | 2026-
11-02 08:45:00+00 |       4.25
            13 |         5 | CG301         | A321      | BLR → MAA | CAPTAIN       | 2026-11-04 00:30:00+00 | 2026-
11-04 03:10:00+00 |       2.67
            15 |         6 | CG302         | A321      | MAA → BLR | CAPTAIN       | 2026-11-04 13:00:00+00 | 2026-
11-04 15:40:00+00 |       2.67
            53 |        21 | CG101         | A320      | BLR → DEL | CAPTAIN       | 2026-11-10 00:00:00+00 | 2026-
11-10 04:15:00+00 |       4.25
            56 |        22 | CG102         | A320      | DEL → BLR | CAPTAIN       | 2026-11-10 15:00:00+00 | 2026-
11-10 19:15:00+00 |       4.25
            59 |        23 | CG301         | A321      | BLR → MAA | CAPTAIN       | 2026-11-12 00:30:00+00 | 2026-
11-12 03:10:00+00 |       2.67
            61 |        24 | CG302         | A321      | MAA → BLR | CAPTAIN       | 2026-11-12 14:00:00+00 | 2026-
11-12 16:40:00+00 |       2.67
(8 rows)
```

Q10 EXISTS + range overlap (&&): who could be added to flight 21, and warnings

EXISTS with the range overlap operator.

```
-- Q10 EXISTS + range overlap (&&): who could be added to flight 21, and warnings

SELECT cm.employee_code, cm.crew_role,

       cm.crew_role = 'CABIN_CREW' OR EXISTS (

         SELECT 1 FROM certification c

         WHERE c.crew_id = cm.crew_id AND c.type_code = f.type_code

           AND f.sched_dep_utc::date BETWEEN c.issued_on AND c.expires_on) AS has_rating,

       EXISTS (

         SELECT 1 FROM duty_assignment d

         WHERE d.crew_id = cm.crew_id

           AND tstzrange(d.duty_start_utc, d.duty_end_utc)

               && tstzrange(f.sched_dep_utc - INTERVAL '1 hour', f.sched_arr_utc + INTERVAL '30 minutes')) AS busy

FROM crew_member cm

CROSS JOIN flight f

WHERE f.flight_id = 21

  AND cm.is_active

  AND cm.crew_id NOT IN (SELECT crew_id FROM duty_assignment WHERE flight_id = 21);

 employee_code |   crew_role   | has_rating | busy
```

```
---------------+---------------+------------+------
... (10 more lines)
```

Q12 Index use: the composite index idx_duty_crew_start serves this lookup

The composite index serving a crew lookup.

```
-- Q12 Index use: the composite index idx_duty_crew_start serves this lookup

-- (on a table this small the planner may still prefer a sequential scan;

--  SET enable_seqscan = off shows the index plan for the demo)


EXPLAIN ANALYZE

SELECT * FROM duty_assignment WHERE crew_id = 1 ORDER BY duty_start_utc;

                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------
-----------------
 Index Scan using idx_duty_crew_start on duty_assignment  (cost=0.14..8.28 rows=8 width=50) (actual time=0.006..0.007
rows=8 loops=1)
   Index Cond: (crew_id = 1)
 Planning Time: 0.060 ms
 Execution Time: 0.018 ms
(4 rows)




-- ---------------------------------------------------------------------

-- PART 2 : COMPLIANCE
```

Q13 Stored procedure: run the validator. BEGIN ... ROLLBACK runs it for real

The validator, called inside a transaction.

```
-- Q13 Stored procedure: run the validator. BEGIN ... ROLLBACK runs it for real

-- and then undoes everything, so the data stays clean. Q14 to Q16 read this run.


CALL sp_run_validation(2, '2026-11-02', '2026-11-15', NULL);

 p_run_id
----------
       21
(1 row)
```

Q14 The explanation: which duties caused each violation

The explanation, through the many-to-many bridge. The output is cut short here.

```
-- Q14 The explanation: which duties caused each violation

-- (violation -> violation_assignment -> v_duty_roster, the M:N bridge)

SELECT v.violation_id, v.rule_code, cm.first_name || ' ' || cm.last_name AS crew_name,

       r.flight_number, r.duty_start_utc, r.duty_end_utc, va.contribution_value AS hours

FROM violation v

JOIN crew_member          cm ON cm.crew_id       = v.crew_id

JOIN violation_assignment va ON va.violation_id  = v.violation_id

JOIN v_duty_roster        r  ON r.assignment_id  = va.assignment_id

WHERE v.run_id = (SELECT MAX(run_id) FROM validation_run)

ORDER BY v.violation_id, r.duty_start_utc;

 violation_id | rule_code |  crew_name   | flight_number |     duty_start_utc     |      duty_end_utc      | hours
--------------+-----------+--------------+---------------+------------------------+------------------------+-------
          250 | RST       | Arjun Menon  | CG101         | 2026-11-02 00:00:00+00 | 2026-11-02 04:15:00+00 |  4.25
          250 | RST       | Arjun Menon  | CG102         | 2026-11-02 04:30:00+00 | 2026-11-02 08:45:00+00 |  4.25
          251 | RST       | Arjun Menon  | CG301         | 2026-11-04 00:30:00+00 | 2026-11-04 03:10:00+00 |  2.67
          251 | RST       | Arjun Menon  | CG302         | 2026-11-04 13:00:00+00 | 2026-11-04 15:40:00+00 |  2.67
          252 | RST       | Rohan Kumar  | CG101         | 2026-11-02 00:00:00+00 | 2026-11-02 04:15:00+00 |  4.25
          252 | RST       | Rohan Kumar  | CG102         | 2026-11-02 04:30:00+00 | 2026-11-02 08:45:00+00 |  4.25
          253 | RST       | Vikram Joshi | CG101         | 2026-11-02 00:00:00+00 | 2026-11-02 04:15:00+00 |  4.25
```

```
          253 | RST       | Vikram Joshi | CG102         | 2026-11-02 04:30:00+00 | 2026-11-02 08:45:00+00 |  4.25
          254 | RST       | Vikram Joshi | CG301         | 2026-11-04 00:30:00+00 | 2026-11-04 03:10:00+00 |  2.67
          254 | RST       | Vikram Joshi | CG302         | 2026-11-04 13:00:00+00 | 2026-11-04 15:40:00+00 |  2.67
          255 | RST       | Aditya Reddy | CG401         | 2026-11-04 17:00:00+00 | 2026-11-04 22:00:00+00 |  5.00
          255 | RST       | Aditya Reddy | CG402         | 2026-11-05 00:00:00+00 | 2026-11-05 05:00:00+00 |  5.00
... (27 more lines)
```

Q15 GROUP BY: violations per rule and severity (dashboard, SQL panel)

Violations per rule and severity.

```
-- Q15 GROUP BY: violations per rule and severity (dashboard, SQL panel)

SELECT rule_code, severity, COUNT(*) AS violations

FROM violation

WHERE run_id = (SELECT MAX(run_id) FROM validation_run)

GROUP BY rule_code, severity

ORDER BY violations DESC;

 rule_code | severity | violations
-----------+----------+------------
 RST       | HIGH     |          7
 CER       | CRITICAL |          4
 WDH       | MEDIUM   |          3
 FDP       | CRITICAL |          2
(4 rows)
```

Q16 GROUP BY + HAVING: crew with more than one violation

HAVING over the report view.

```
-- Q16 GROUP BY + HAVING: crew with more than one violation

SELECT crew_name, COUNT(*) AS violations, STRING_AGG(DISTINCT rule_code, ', ') AS rules

FROM v_violation_report

WHERE run_id = (SELECT MAX(run_id) FROM validation_run)

GROUP BY crew_id, crew_name

HAVING COUNT(*) > 1

ORDER BY violations DESC, crew_name;

  crew_name   | violations |  rules
--------------+------------+----------
 Rohan Kumar  |          3 | CER, RST
 Aditya Reddy |          2 | FDP, RST
 Arjun Menon  |          2 | RST
 Priya Iyer   |          2 | FDP, RST
 Sneha Patil  |          2 | CER
 Vikram Joshi |          2 | RST
(6 rows)
```

Q17 The review workflow: the trigger stamps reviewed_at, the CHECK needs a note

The review workflow.

```
-- Q17 The review workflow: the trigger stamps reviewed_at, the CHECK needs a note

UPDATE violation

SET review_status = 'RESOLVED', review_note = 'Crew swapped on the roster', reviewed_by = 2

WHERE violation_id = (SELECT MIN(violation_id) FROM violation WHERE run_id = (SELECT MAX(run_id) FROM
validation_run))

RETURNING violation_id, review_status, reviewed_at;

 violation_id | review_status |          reviewed_at
--------------+---------------+-------------------------------
          250 | RESOLVED      | 2026-11-02 17:11:35.979199+00
(1 row)

UPDATE 1
```

Q18 Full-text search: ranked rule search with a highlighted snippet (rules page)

Ranked full-text search with a highlighted snippet.

```
-- Q18 Full-text search: ranked rule search with a highlighted snippet (rules page)

SELECT rule_code, rule_name,

       ts_rank(to_tsvector('english', rule_name || ' ' || description || ' ' || COALESCE(regulation_ref, '')),

               websearch_to_tsquery('english', 'rest period')) AS rank,

       ts_headline('english', description, websearch_to_tsquery('english', 'rest period')) AS snippet

FROM rule_definition

WHERE to_tsvector('english', rule_name || ' ' || description || ' ' || COALESCE(regulation_ref, ''))

      @@ websearch_to_tsquery('english', 'rest period')

ORDER BY rank DESC;

 rule_code |          rule_name          |    rank    |                                    snippet
-----------+-----------------------------+------------+-----------------------------------------------------------------
-----------------------
 RST       | Minimum Rest Between Duties | 0.11868033 | <b>rest</b> <b>period</b> between the end of one duty and the
start of the next duty
(1 row)
```

Q19 Index use: the same search through the GIN index idx_rule_fts

The search going through the GIN index.

```
-- Q19 Index use: the same search through the GIN index idx_rule_fts

-- (four rows are too few for the planner to choose it on its own)


EXPLAIN ANALYZE

SELECT rule_code FROM rule_definition

WHERE to_tsvector('english', rule_name || ' ' || description || ' ' || COALESCE(regulation_ref, ''))

      @@ websearch_to_tsquery('english', 'rest');

                                                                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------
 Bitmap Heap Scan on rule_definition  (cost=12.81..17.08 rows=1 width=4) (actual time=0.029..0.029 rows=1 loops=1)
   Recheck Cond: (to_tsvector('english'::regconfig, (((((rule_name)::text || ' '::text) || description) || ' '::text)
|| (COALESCE(regulation_ref, ''::character varying))::text)) @@ '''rest'''::tsquery)
   Heap Blocks: exact=1
   ->  Bitmap Index Scan on idx_rule_fts  (cost=0.00..12.81 rows=1 width=0) (actual time=0.023..0.023 rows=1 loops=1)
         Index Cond: (to_tsvector('english'::regconfig, (((((rule_name)::text || ' '::text) || description) || '
'::text) || (COALESCE(regulation_ref, ''::character varying))::text)) @@ '''rest'''::tsquery)
 Planning Time: 0.042 ms
 Execution Time: 0.046 ms
(7 rows)
```

8. System architecture

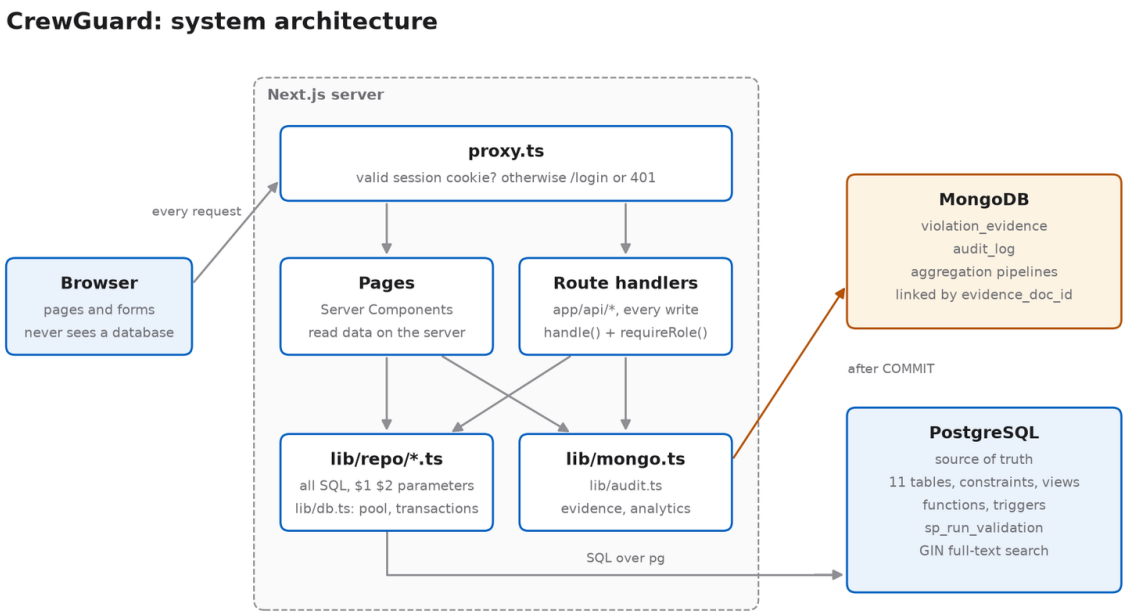


Figure 3: System architecture.

The browser never connects to a database. `proxy.ts` runs first on every request and sends anyone without a valid session to the login page, or returns 401 for the API. Pages are Server Components that read through `lib/repo`. Every write goes through a route handler, which checks the role and then calls the repository.

Concern	How it is handled
SQL injection	All SQL is parameterised with `$1, $2` and an array of values, so user input is never executed as SQL. There is no ORM, so every query is visible.
Passwords	Only bcrypt hashes are stored. A wrong username and a wrong password give the same message, so usernames cannot be discovered.
Sessions	A JWT signed with `SESSION_SECRET` in an httpOnly cookie, valid for 8 hours. A user cannot edit their own role without breaking the signature.
Authorisation	`requireRole()` in every write route: 401 if not signed in, 403 for the wrong role.
Credentials	In `.env.local`, which is git-ignored. `.env.example` is the committed template.
Times	Flights and duties are stored and shown in UTC. Logins, validation runs and reviews happen on the users' clock, so they are shown in IST and labelled.
Validation errors	When PostgreSQL rejects a row, `handle()` maps the constraint name to a readable message, and the page shows both.

9. Application screenshots

Flight and duty times are shown in UTC. Logins, validation runs and reviews are shown in IST and labelled.

9.1 Signing in

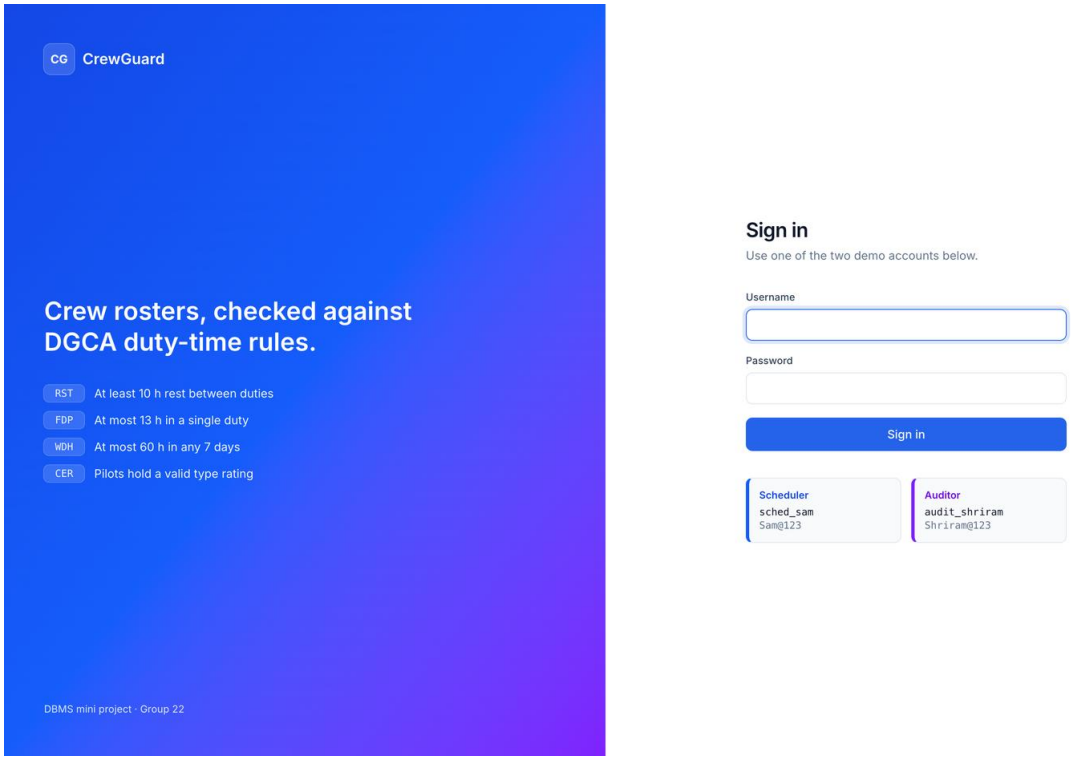


Figure 4: Login page.

9.2 Scheduler: the roster

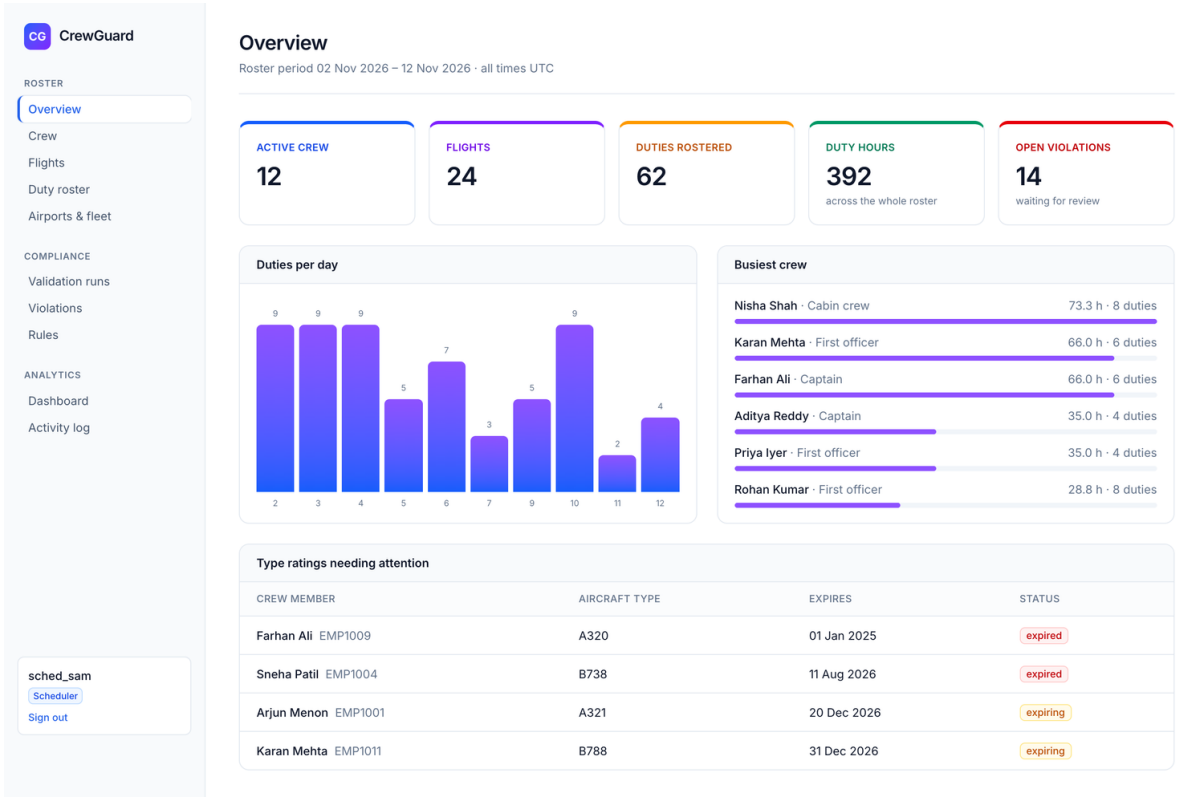


Figure 5: Overview, with the open violations count.

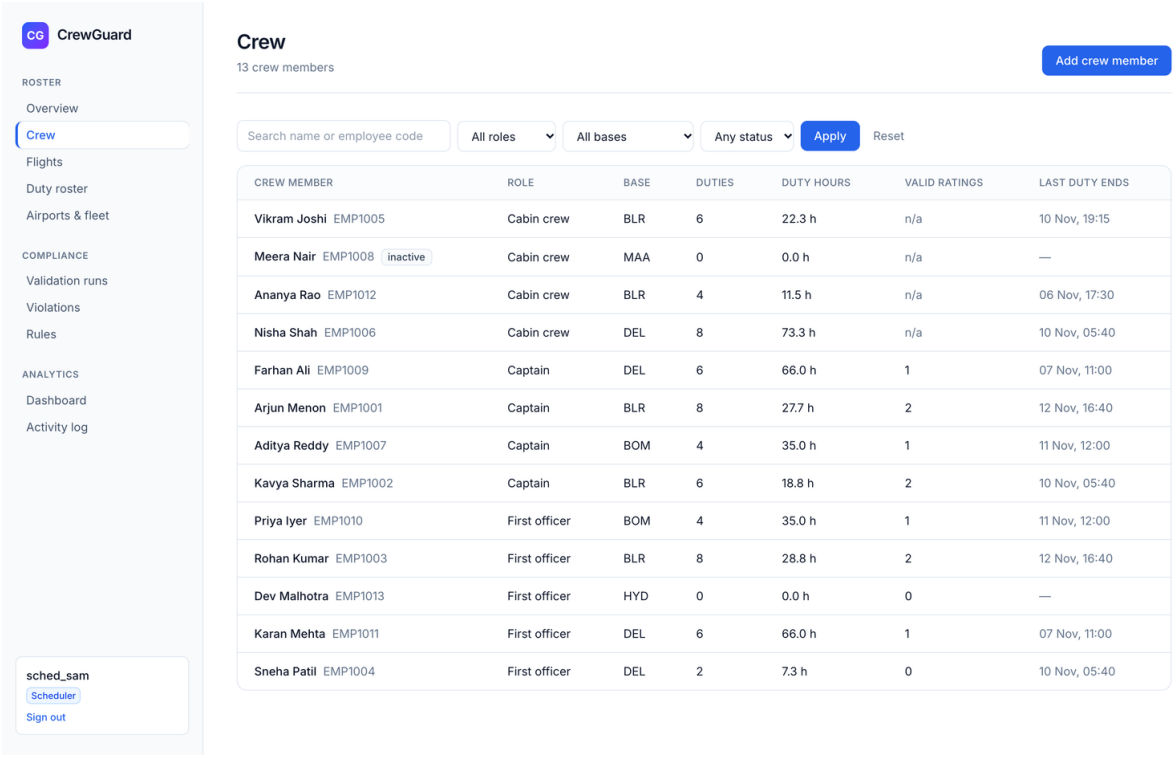


Figure 6: Crew list with search and filters.

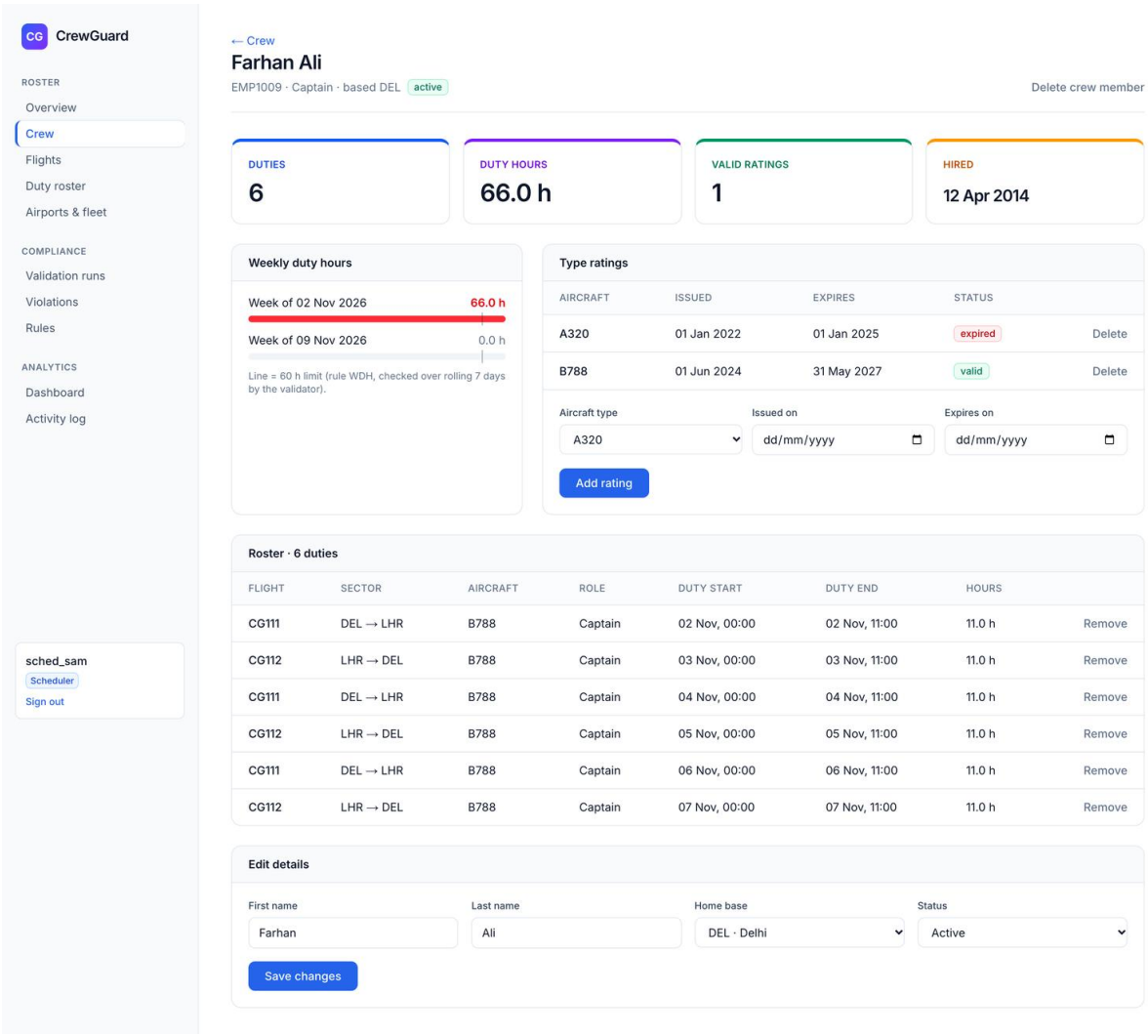


Figure 7: Crew detail: weekly hours against the 60 h line, ratings with their status, and the roster.

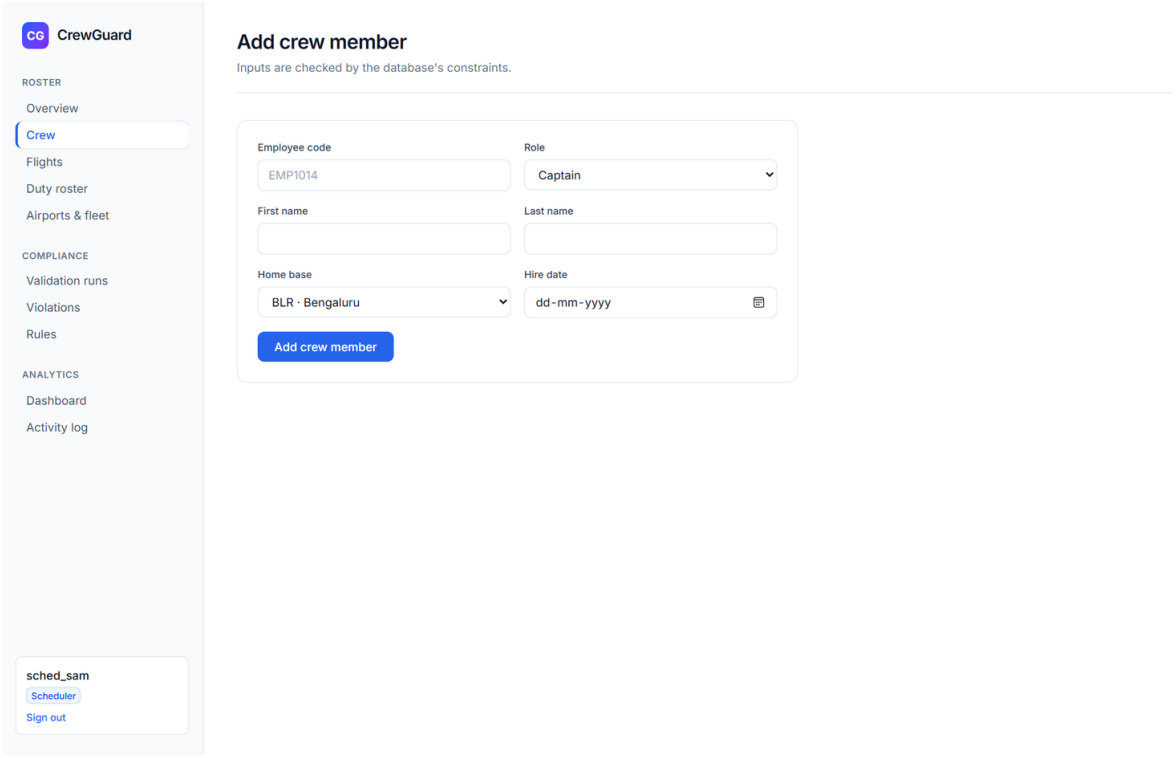


Figure 8: Add crew member.

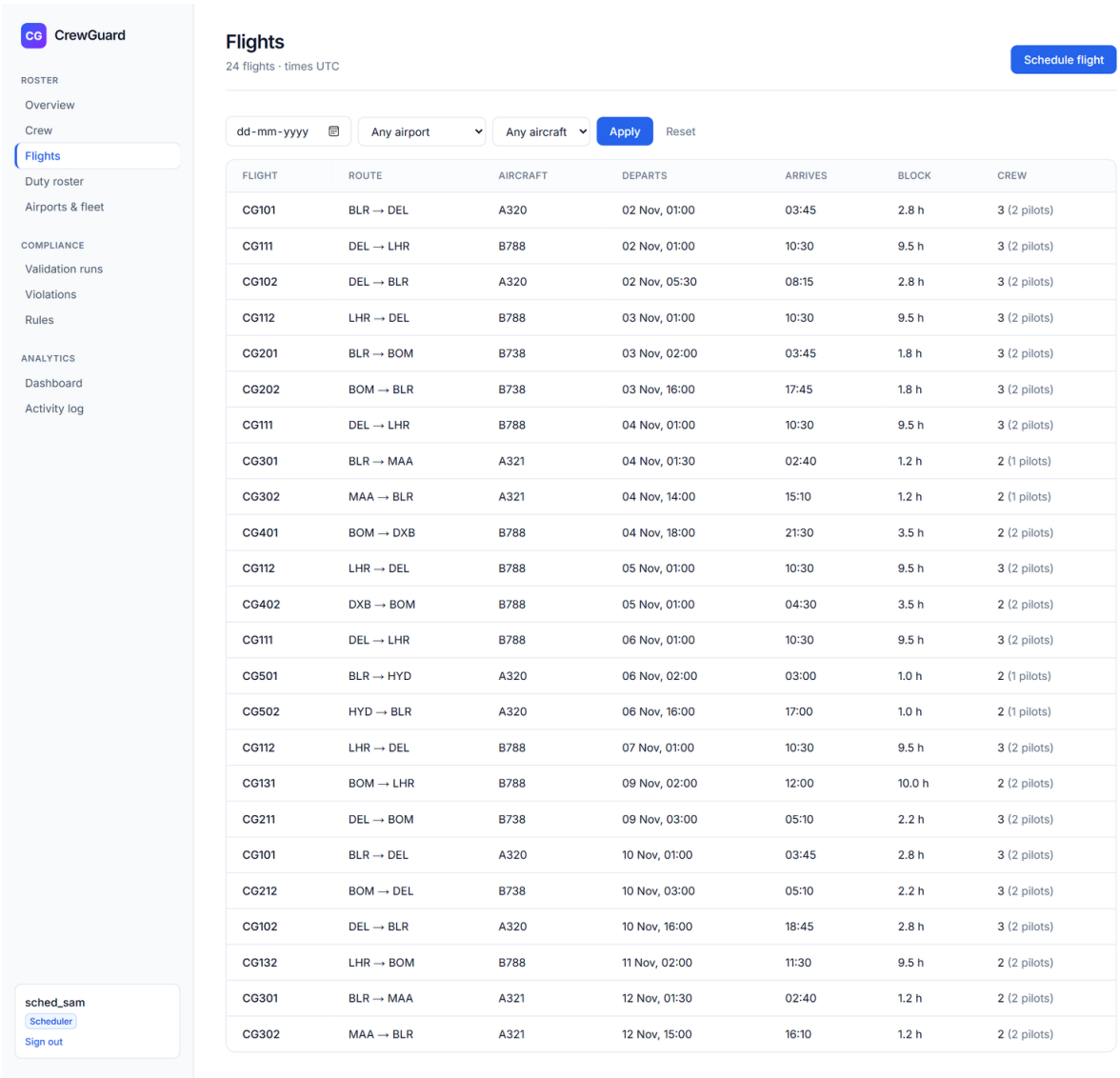


Figure 9: Flights list with crew counts.

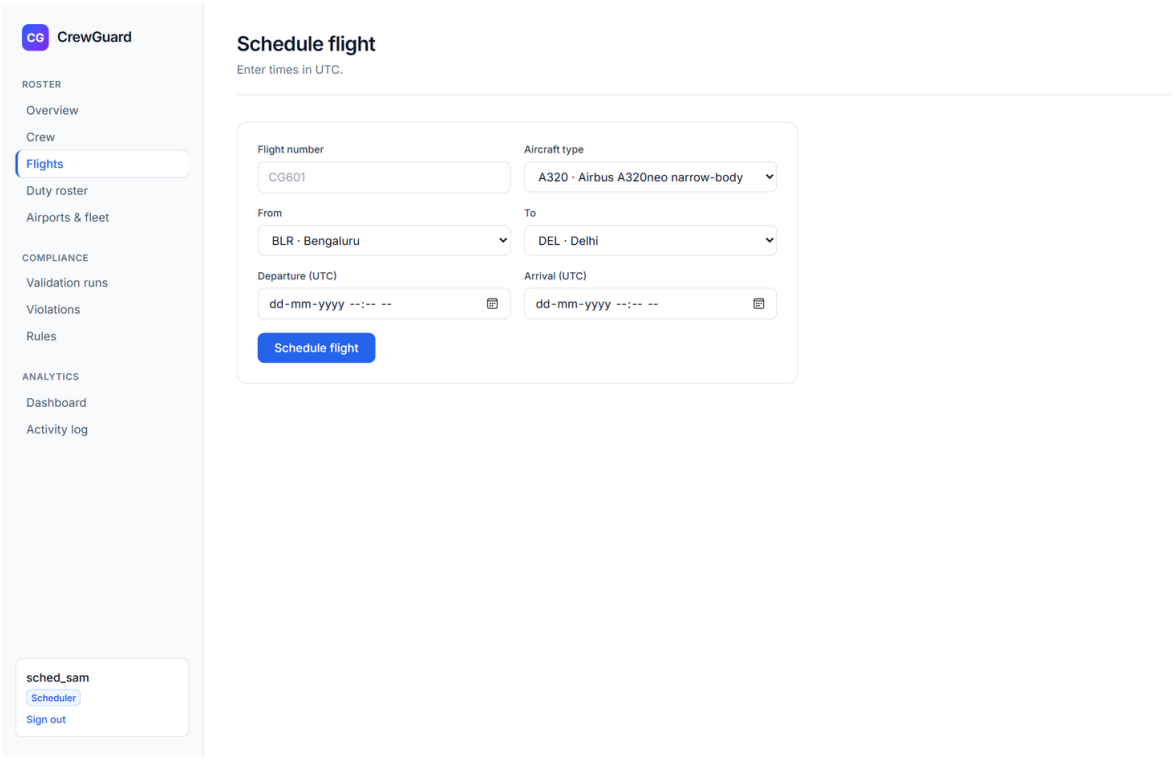


Figure 10: Schedule a flight.

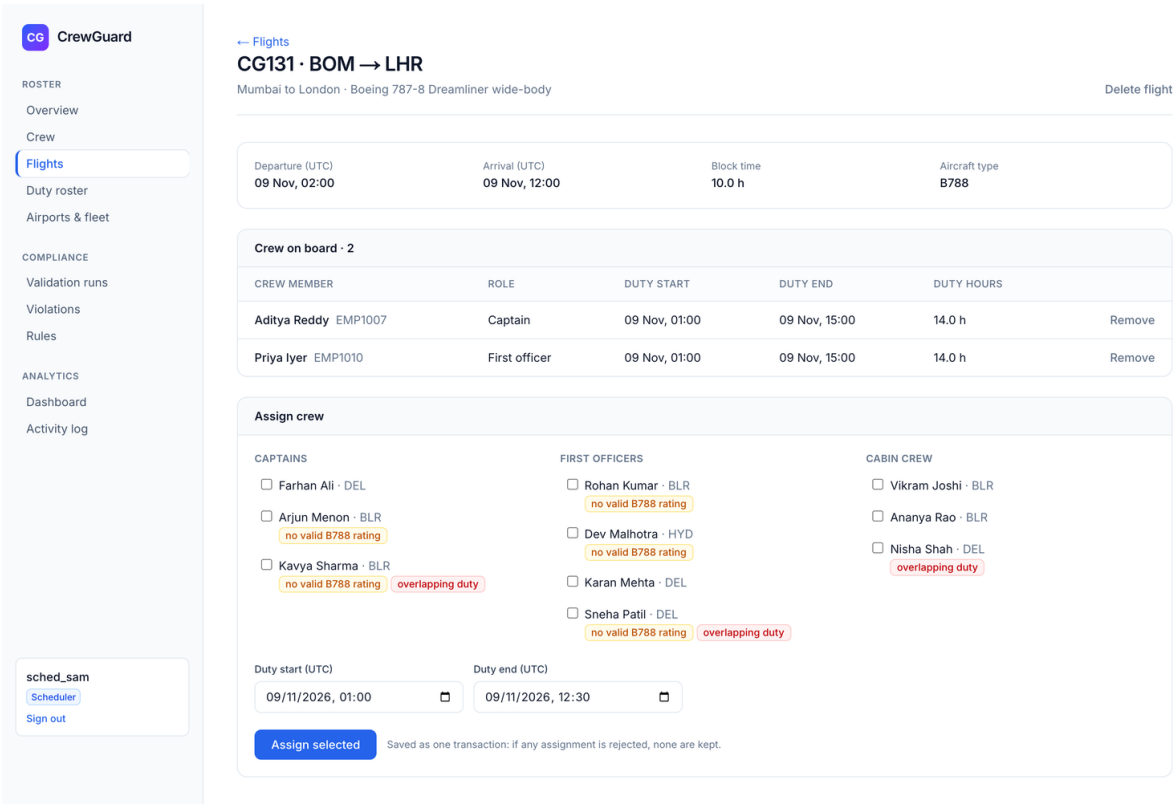


Figure 11: Flight CG131 with the multi-crew assign form and its rating and overlap hints.

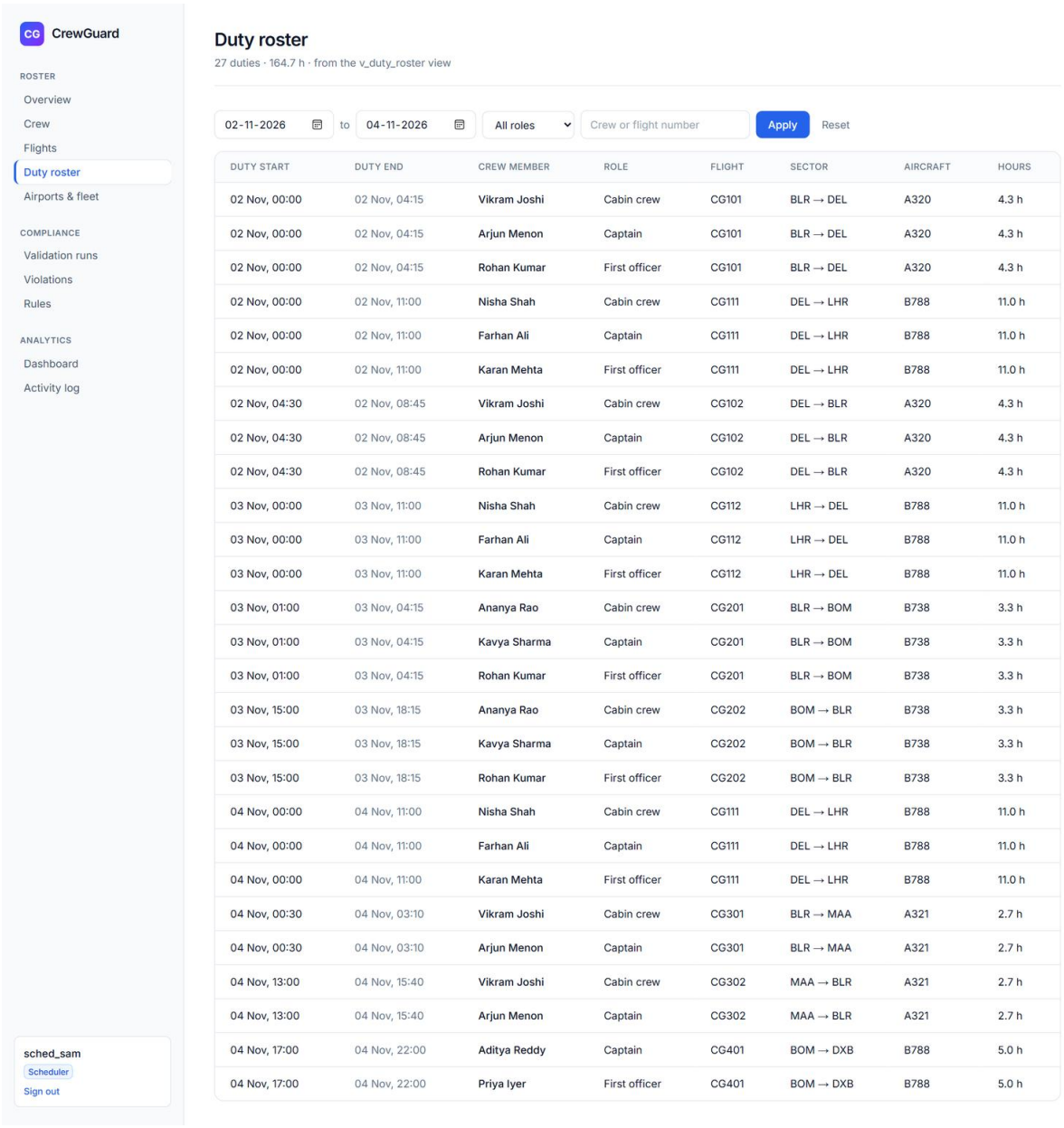


Figure 12: Duty roster from the v_duty_roster view, filtered to 2 to 4 November.

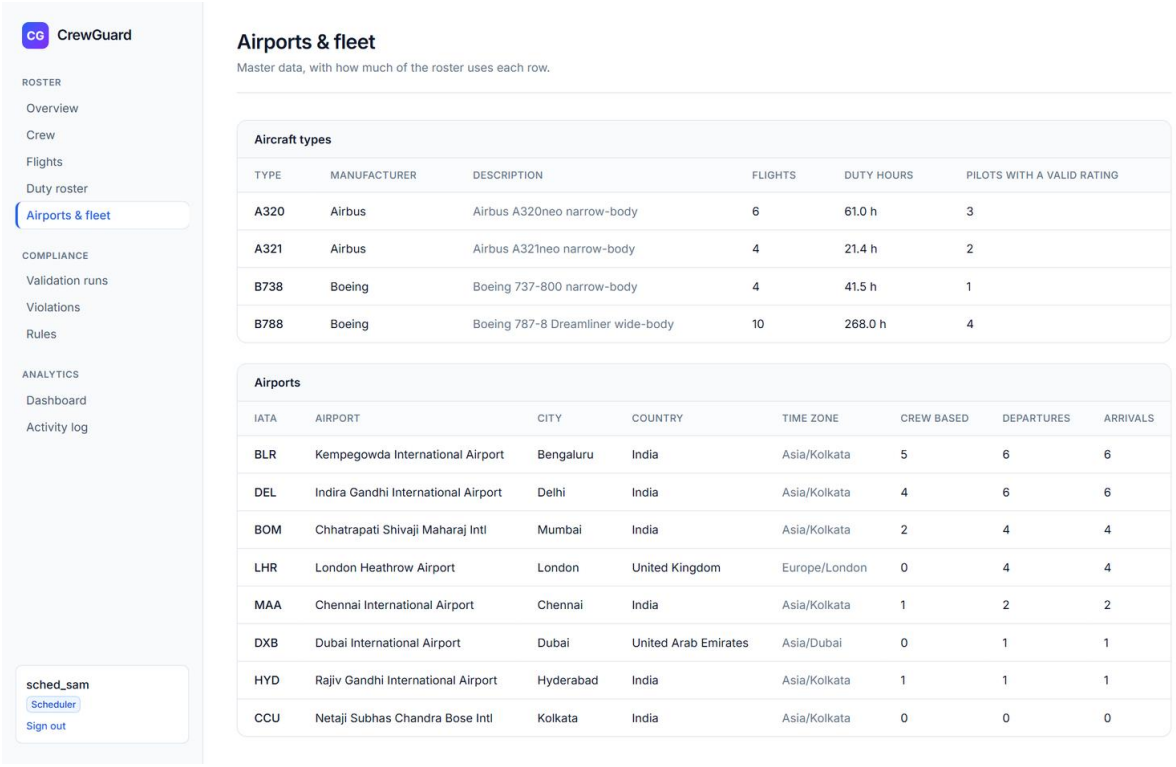


Figure 13: Airports and fleet.

9.3 The database rejecting bad input

These are real submissions. In each case PostgreSQL rejected the write, and the page shows the message and the name of the constraint.

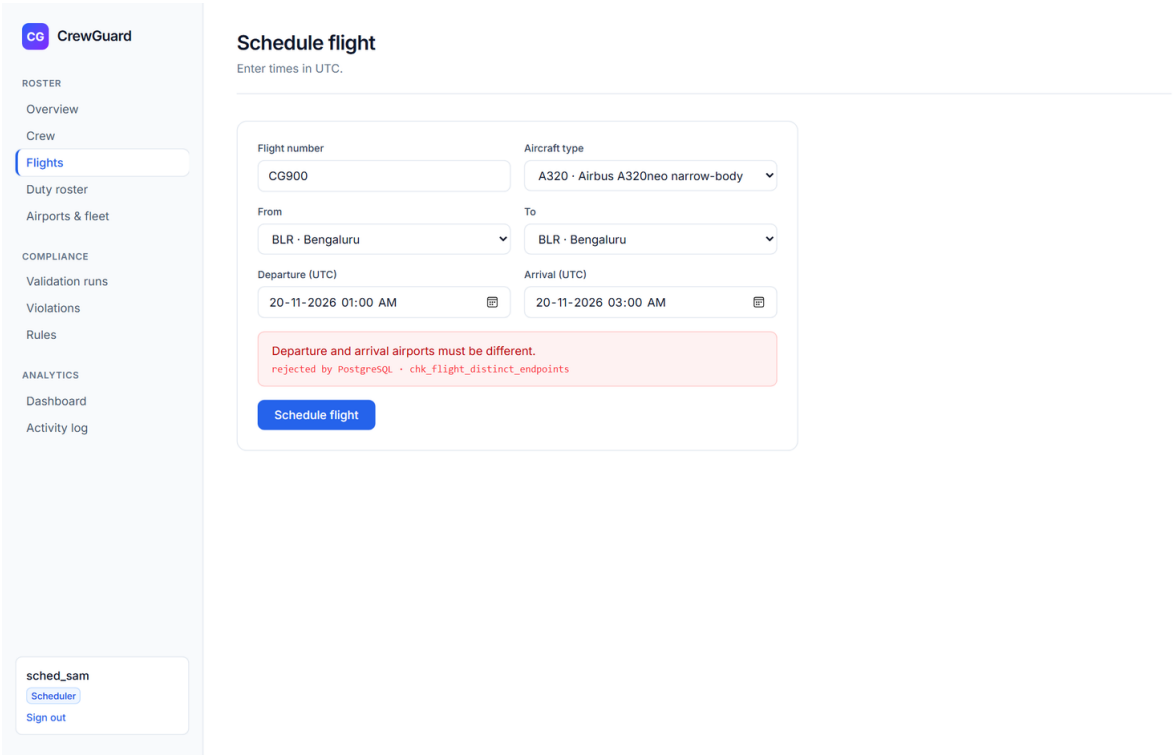


Figure 14: chk_flight_distinct_endpoints: a flight cannot depart and arrive at the same airport.

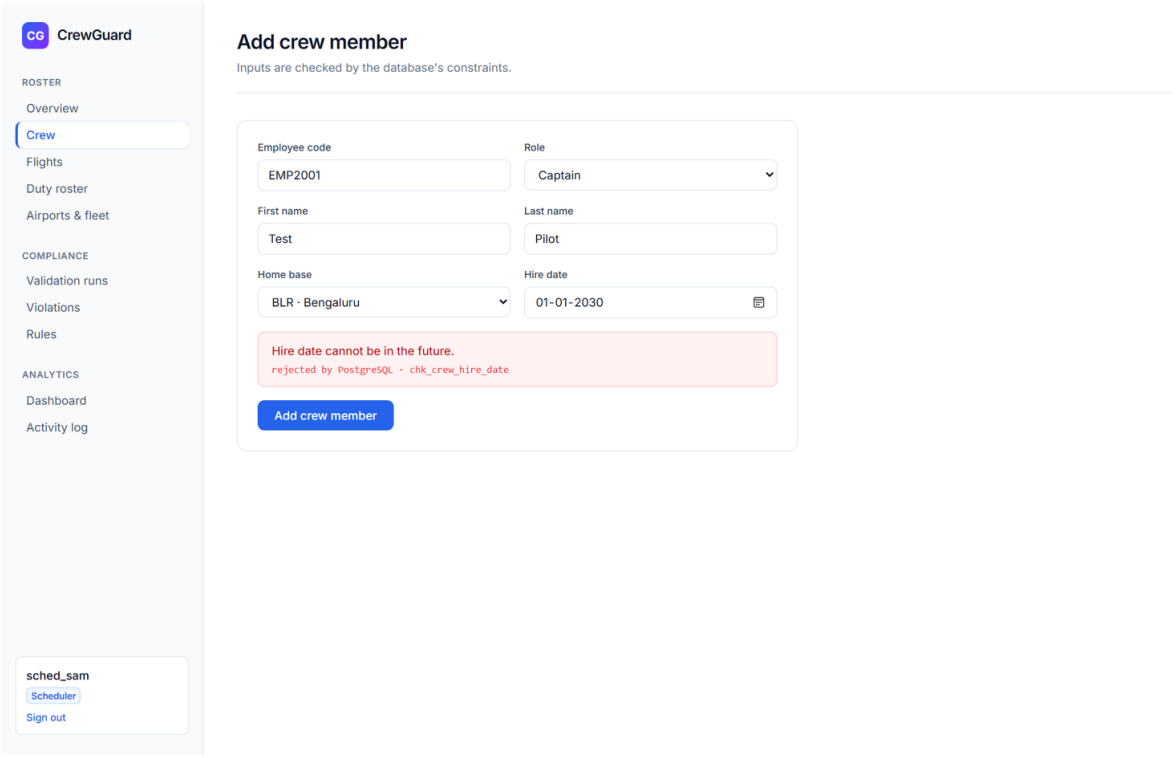


Figure 15: chk_crew_hire_date: a hire date cannot be in the future.

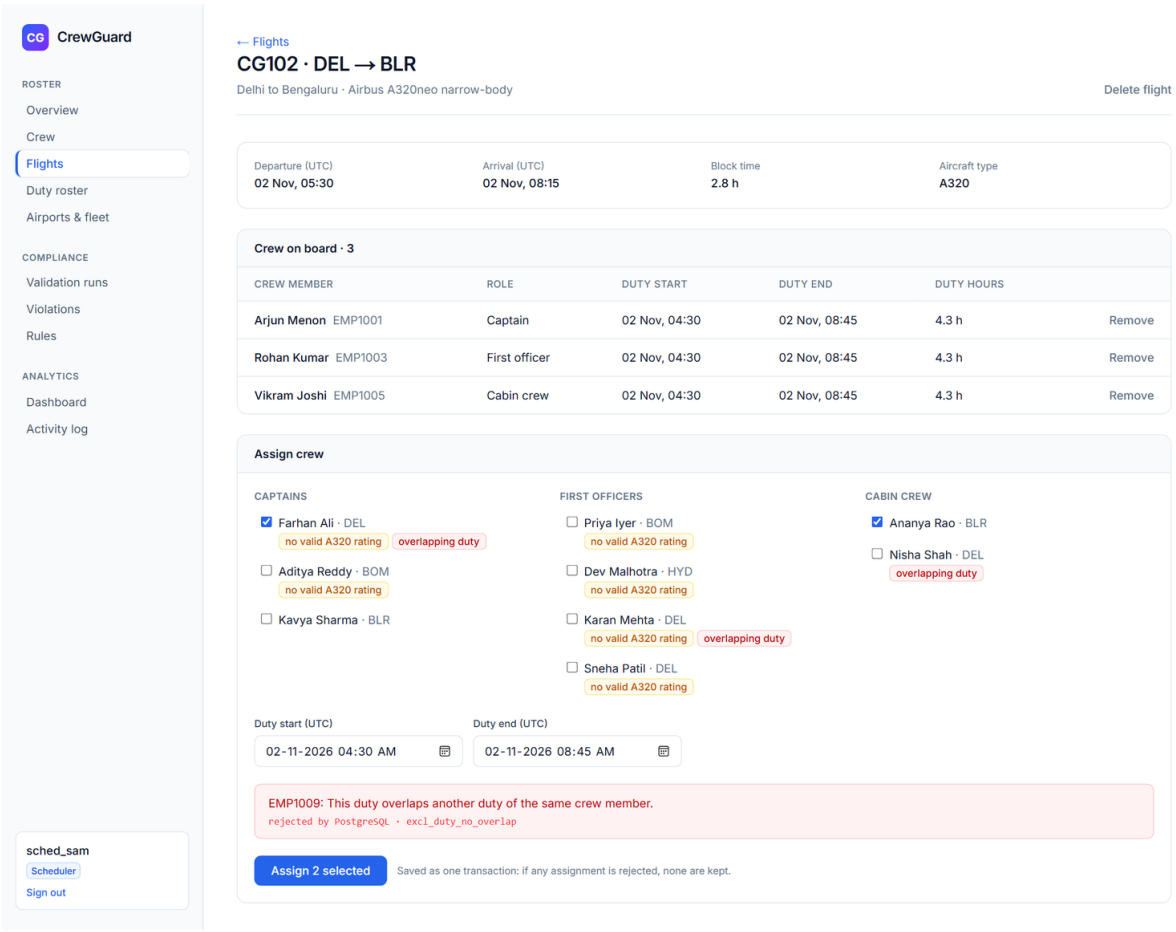


Figure 16: excl_duty_no_overlap inside a transaction: Farhan overlaps, so Ananya is not saved either. Crew on board stays at 3.

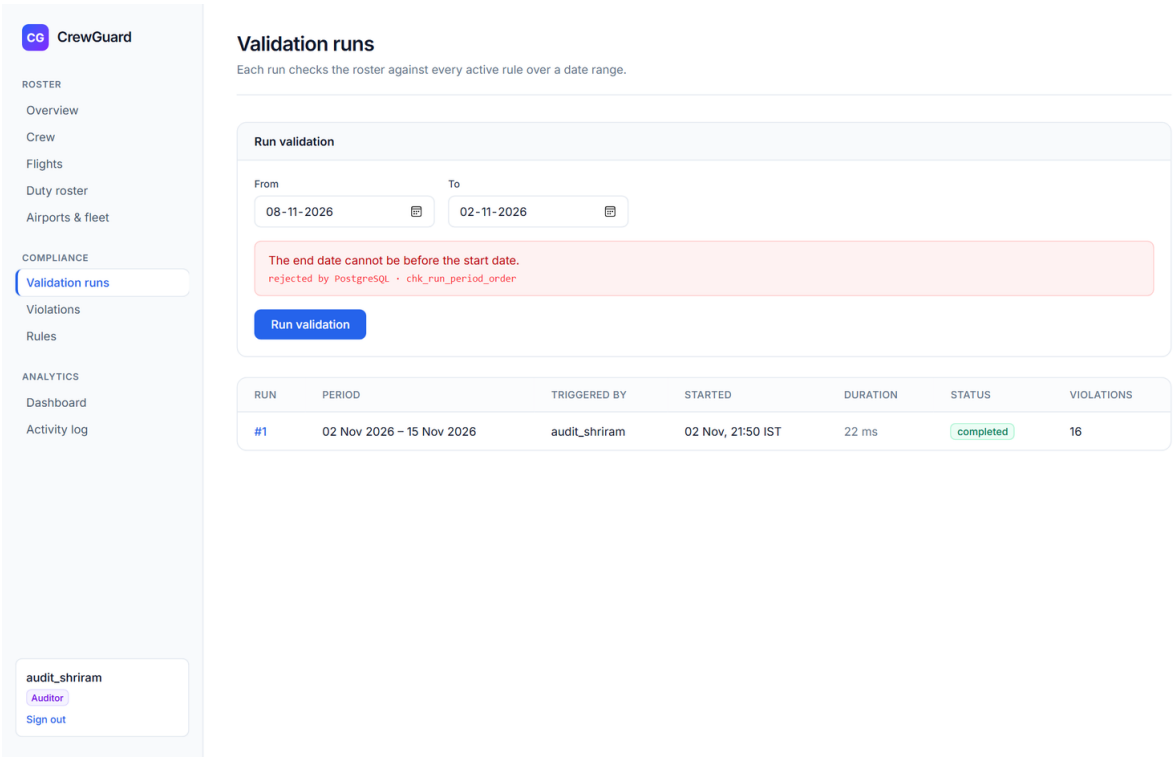


Figure 17: chk_run_period_order: a run cannot end before it starts.

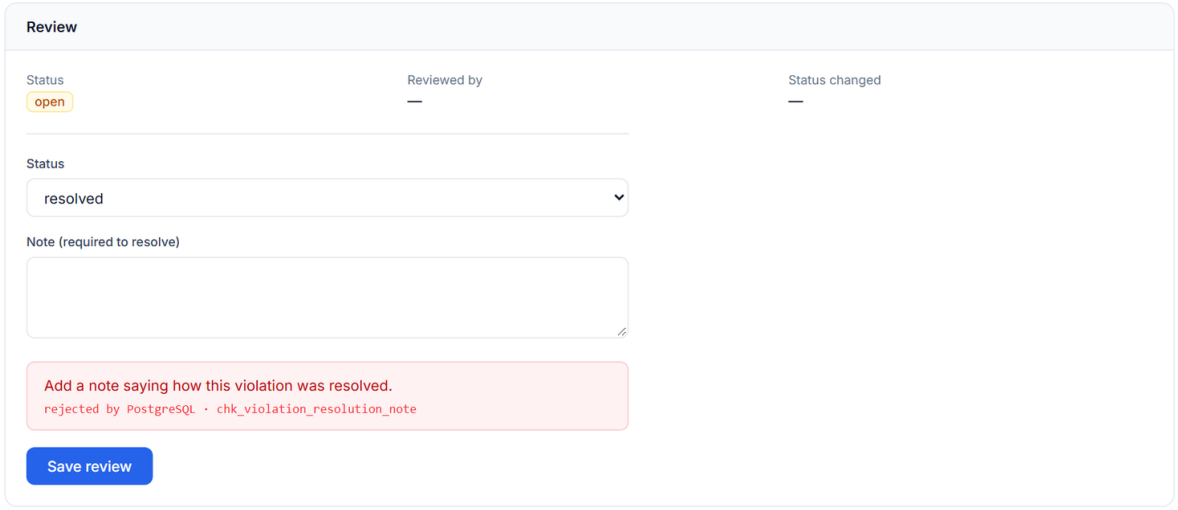


Figure 18: chk_violation_resolution_note: a violation cannot be resolved without a note.

9.4 Auditor: runs and violations

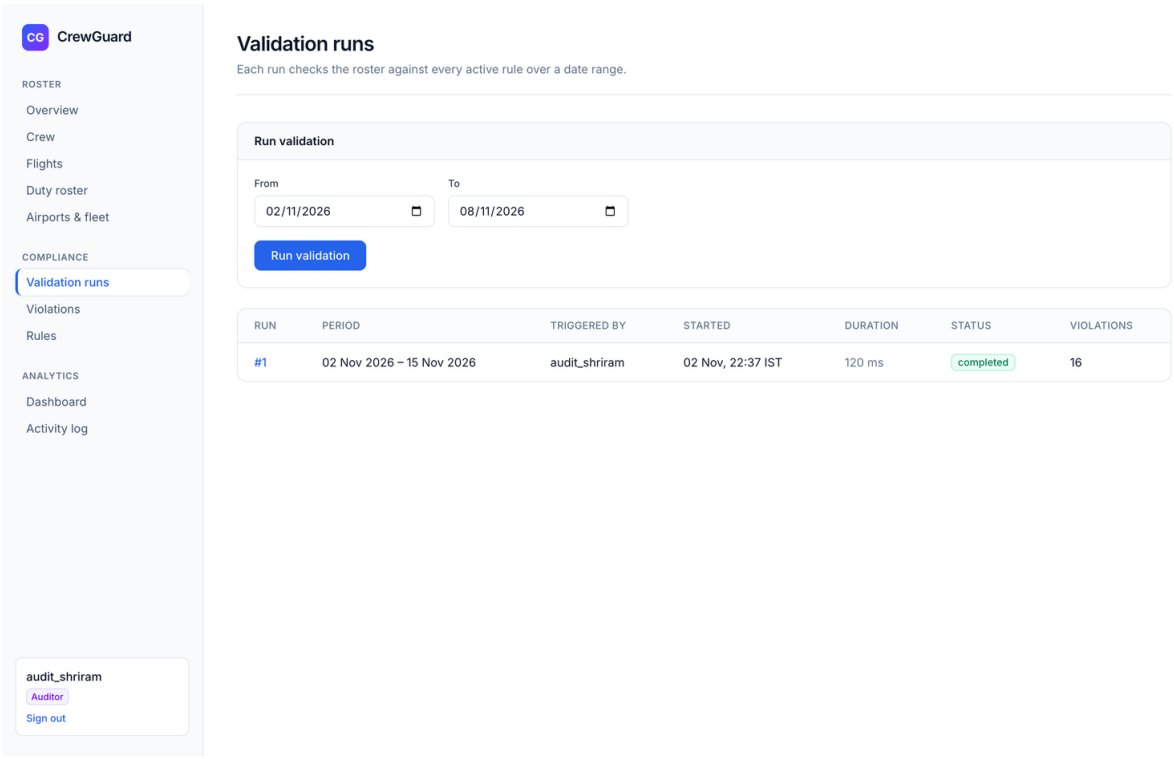


Figure 19: Validation runs, with the form only the auditor sees.

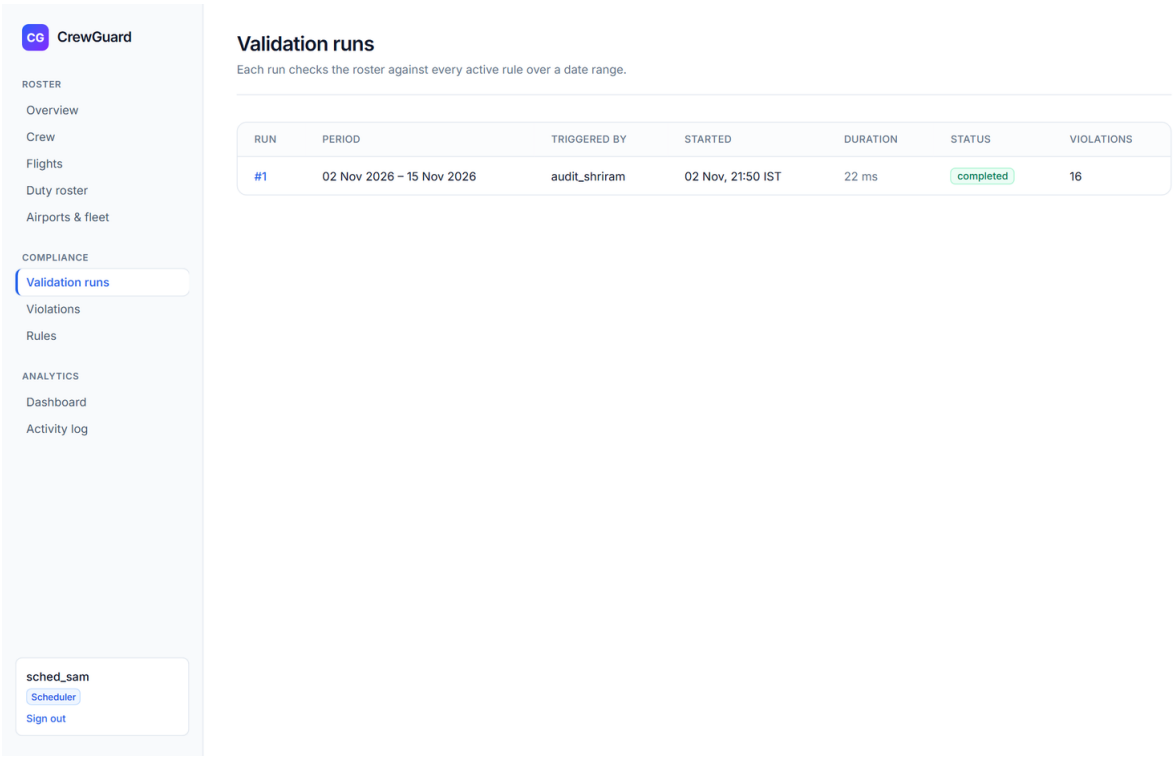


Figure 20: The same page as the scheduler, with no form.

CrewGuard

ROSTER
Overview
Crew
Flights
Duty roster
Airports & fleet

COMPLIANCE
Validation runs
Violations
Rules

ANALYTICS
Dashboard
Activity log

audit_shriram
Auditor
Sign out

← Validation runs

Run #1

02 Nov 2026 – 15 Nov 2026 · 16 violations

Status	Triggered by	Started	Duration
completed	audit_shriram	02 Nov, 22:37 IST	120 ms

CER	FDP	RST	WDH
4	2	7	3
Valid Type Certification Required	Maximum Flight Duty Period	Minimum Rest Between Duties	Maximum Duty Hours in Seven Days

Violations

#	RULE	CREW MEMBER	SEVERITY	OBSERVED	LIMIT	WINDOW START	DUTIES	REVIEW
1	RST	Arjun Menon EMP1001	high	0.25 h	10.00 h	02 Nov, 04:15	2	acknowledged
2	RST	Arjun Menon EMP1001	high	9.83 h	10.00 h	04 Nov, 03:10	2	open
3	RST	Rohan Kumar EMP1003	high	0.25 h	10.00 h	02 Nov, 04:15	2	open
4	RST	Vikram Joshi EMP1005	high	0.25 h	10.00 h	02 Nov, 04:15	2	open
5	RST	Vikram Joshi EMP1005	high	9.83 h	10.00 h	04 Nov, 03:10	2	open
6	RST	Aditya Reddy EMP1007	high	2.00 h	10.00 h	04 Nov, 22:00	2	open
7	RST	Priya Iyer EMP1010	high	2.00 h	10.00 h	04 Nov, 22:00	2	open
8	FDP	Aditya Reddy EMP1007	critical	14.00 h	13.00 h	09 Nov, 01:00	1	resolved
9	FDP	Priya Iyer EMP1010	critical	14.00 h	13.00 h	09 Nov, 01:00	1	open
10	WDH	Nisha Shah EMP1006	medium	66.00 h	60.00 h	31 Oct, 00:00	6	open
11	WDH	Farhan Ali EMP1009	medium	66.00 h	60.00 h	31 Oct, 00:00	6	open
12	WDH	Karan Mehta EMP1011	medium	66.00 h	60.00 h	31 Oct, 00:00	6	open
13	CER	Rohan Kumar EMP1003	critical	—	—	03 Nov, 01:00	1	open
14	CER	Rohan Kumar EMP1003	critical	—	—	03 Nov, 15:00	1	open
15	CER	Sneha Patil EMP1004	critical	—	—	09 Nov, 02:00	1	open
16	CER	Sneha Patil EMP1004	critical	—	—	10 Nov, 02:00	1	open

Figure 21: Run detail for 2 to 15 November: RST 7, FDP 2, WDH 3, CER 4.

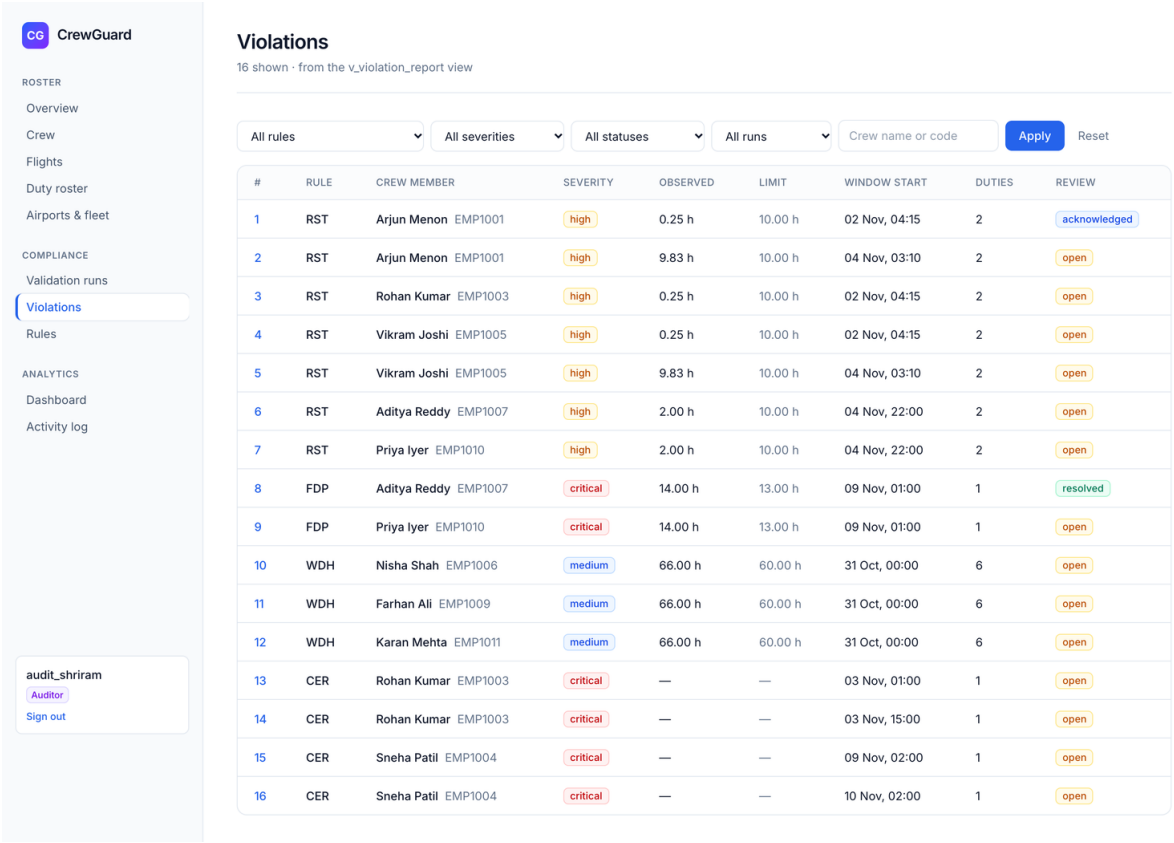


Figure 22: Violations from the v_violation_report view.

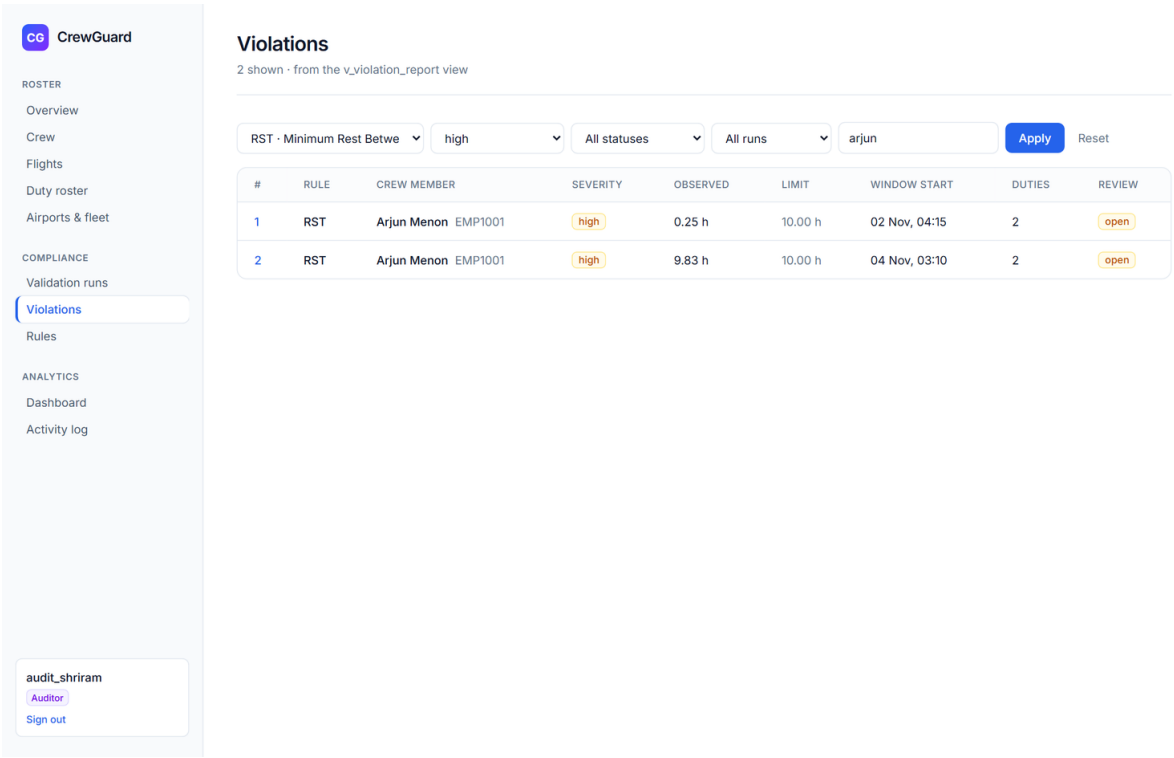


Figure 23: Violations filtered by rule, severity and crew.

9.5 The explanation pages

CG CrewGuard

ROSTER
Overview
Crew
Flights
Duty roster
Airports & fleet

COMPLIANCE
Validation runs
Violations
Rules

ANALYTICS
Dashboard
Activity log

audit_shriram
Auditor
Sign out

← Run #1

WDH · Farhan Ali

EMP1009 · Captain medium open

Maximum Duty Hours in Seven Days

Cumulative duty hours in any rolling window of seven consecutive days must not exceed the prescribed limit, which guards against fatigue that builds up over a week.

Regulation: DGCA CAR Section 7 Series J Part IV

Breach window: 31 Oct, 00:00 to 07 Nov, 11:00

What went wrong

HOURS IN WINDOW	MAXIMUM HOURS
66.00 h	60.00 h

Contributing duties · 6

FLIGHT	SECTOR	AIRCRAFT	DUTY START	DUTY END	HOURS
CG111	DEL → LHR	B788	02 Nov, 00:00	02 Nov, 11:00	11.00 h
CG112	LHR → DEL	B788	03 Nov, 00:00	03 Nov, 11:00	11.00 h
CG111	DEL → LHR	B788	04 Nov, 00:00	04 Nov, 11:00	11.00 h
CG112	LHR → DEL	B788	05 Nov, 00:00	05 Nov, 11:00	11.00 h
CG111	DEL → LHR	B788	06 Nov, 00:00	06 Nov, 11:00	11.00 h
CG112	LHR → DEL	B788	07 Nov, 00:00	07 Nov, 11:00	11.00 h
Total duty hours					66.00 h

Evidence MongoDB · 6ae8c35701b127f991804e42

Window start: 31 Oct, 00:00
Window end: 07 Nov, 11:00
Total in window: 66.00 h
Limit: 60.00 h

FLIGHT	DUTY START	HOURS
CG111	02 Nov, 00:00	11.00 h
CG112	03 Nov, 00:00	11.00 h
CG111	04 Nov, 00:00	11.00 h
CG112	05 Nov, 00:00	11.00 h
CG111	06 Nov, 00:00	11.00 h
CG112	07 Nov, 00:00	11.00 h

Review

Status: open
Reviewed by: —
Status changed: —

Status: open

Note (required to resolve)

Save review

Figure 24: WDH: 66 h against 60 h, with the six duties that add up to it and the MongoDB evidence.

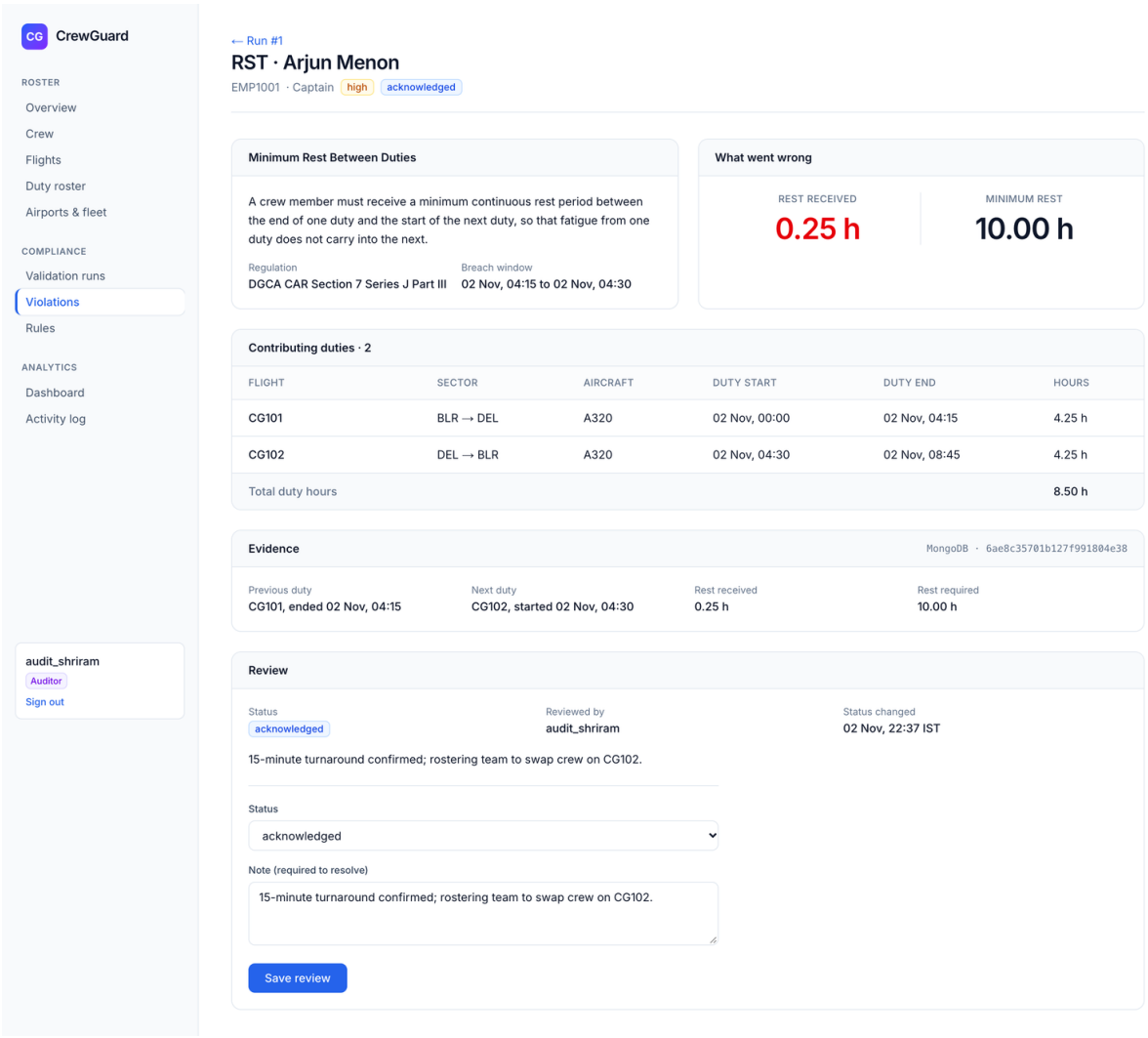


Figure 25: RST: 0.25 h of rest between CG101 and CG102.

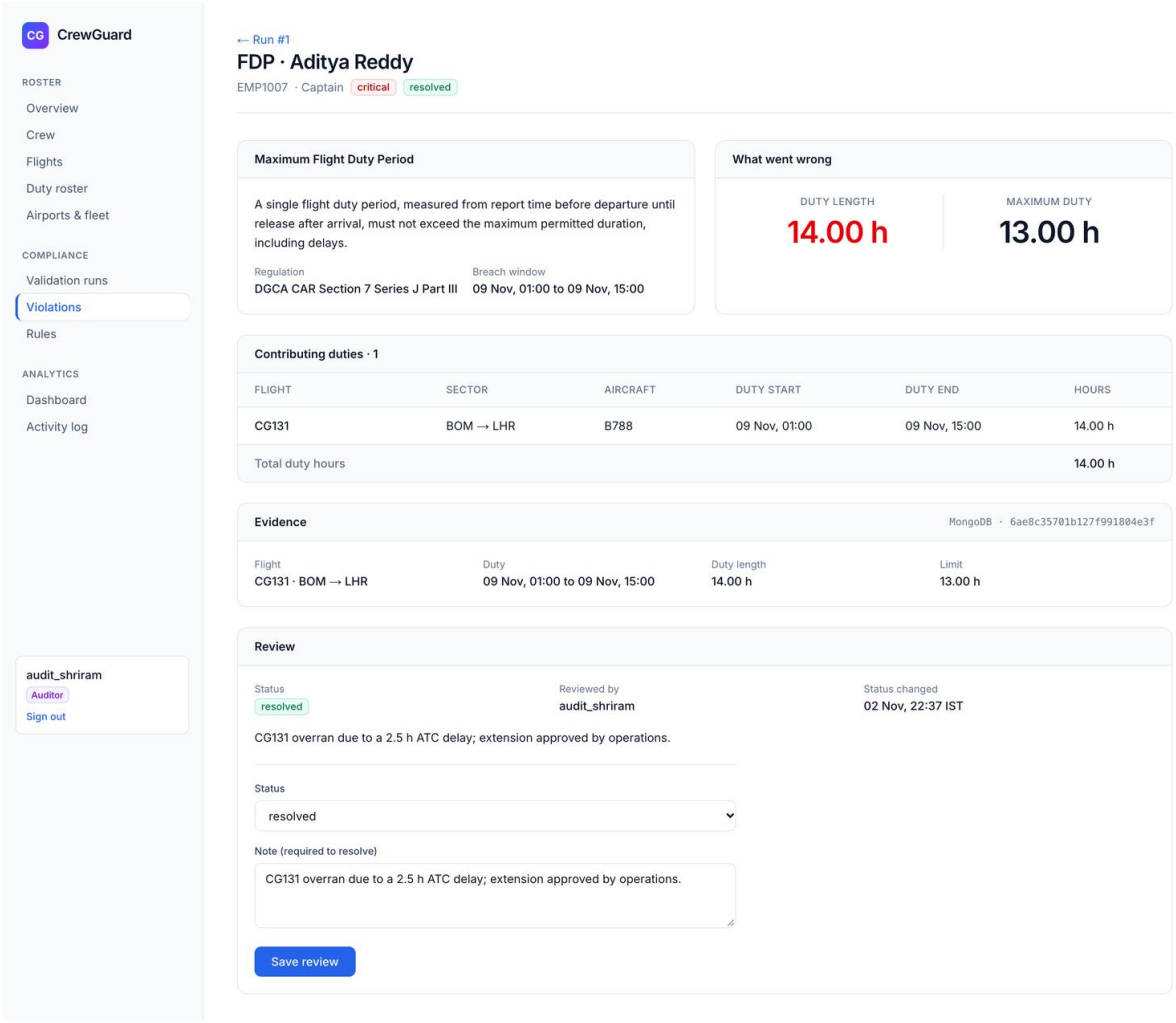


Figure 26: FDP: a delayed flight making a 14 h duty.

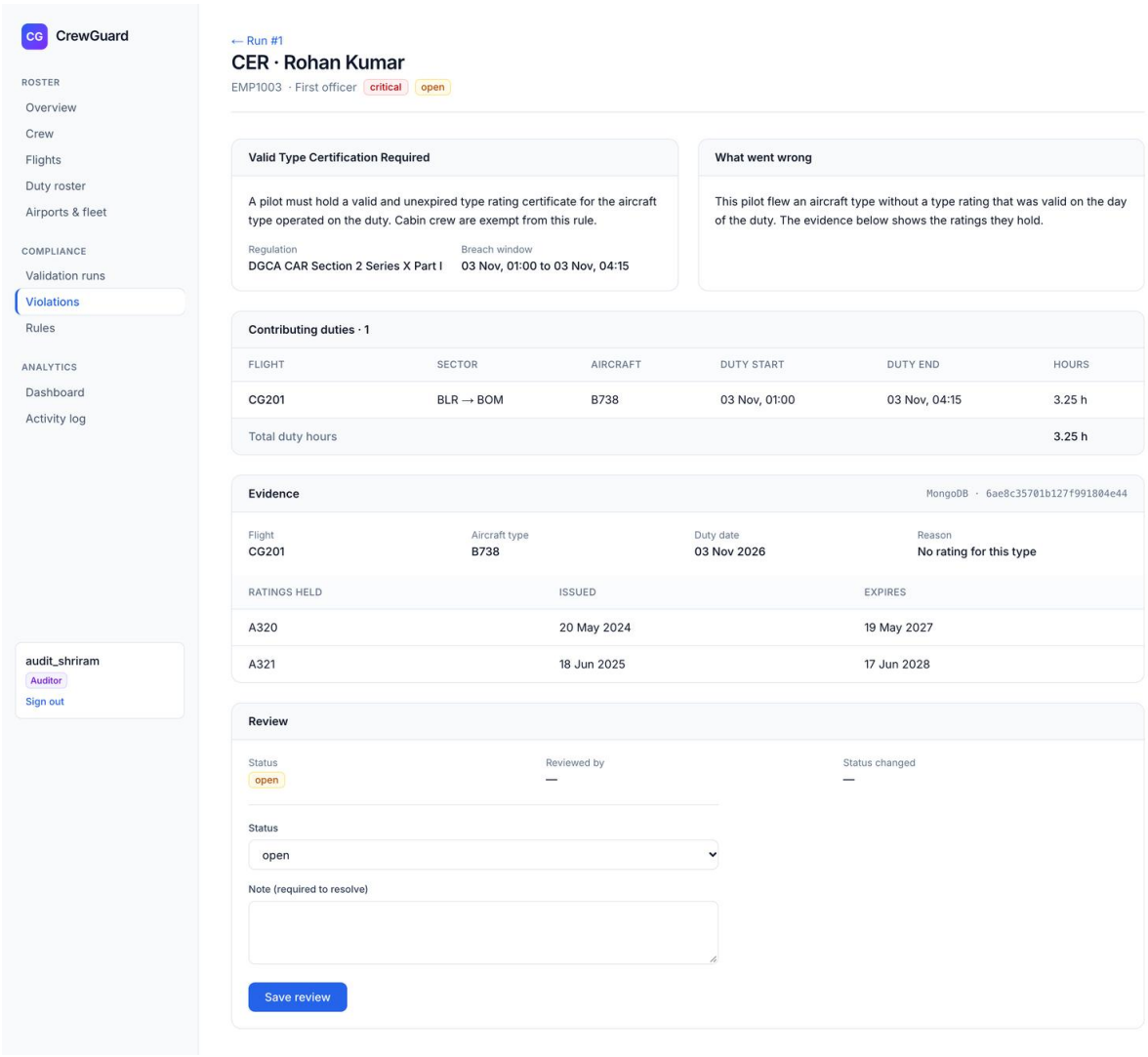


Figure 27: CER: Rohan flew a B738 with no B738 rating.

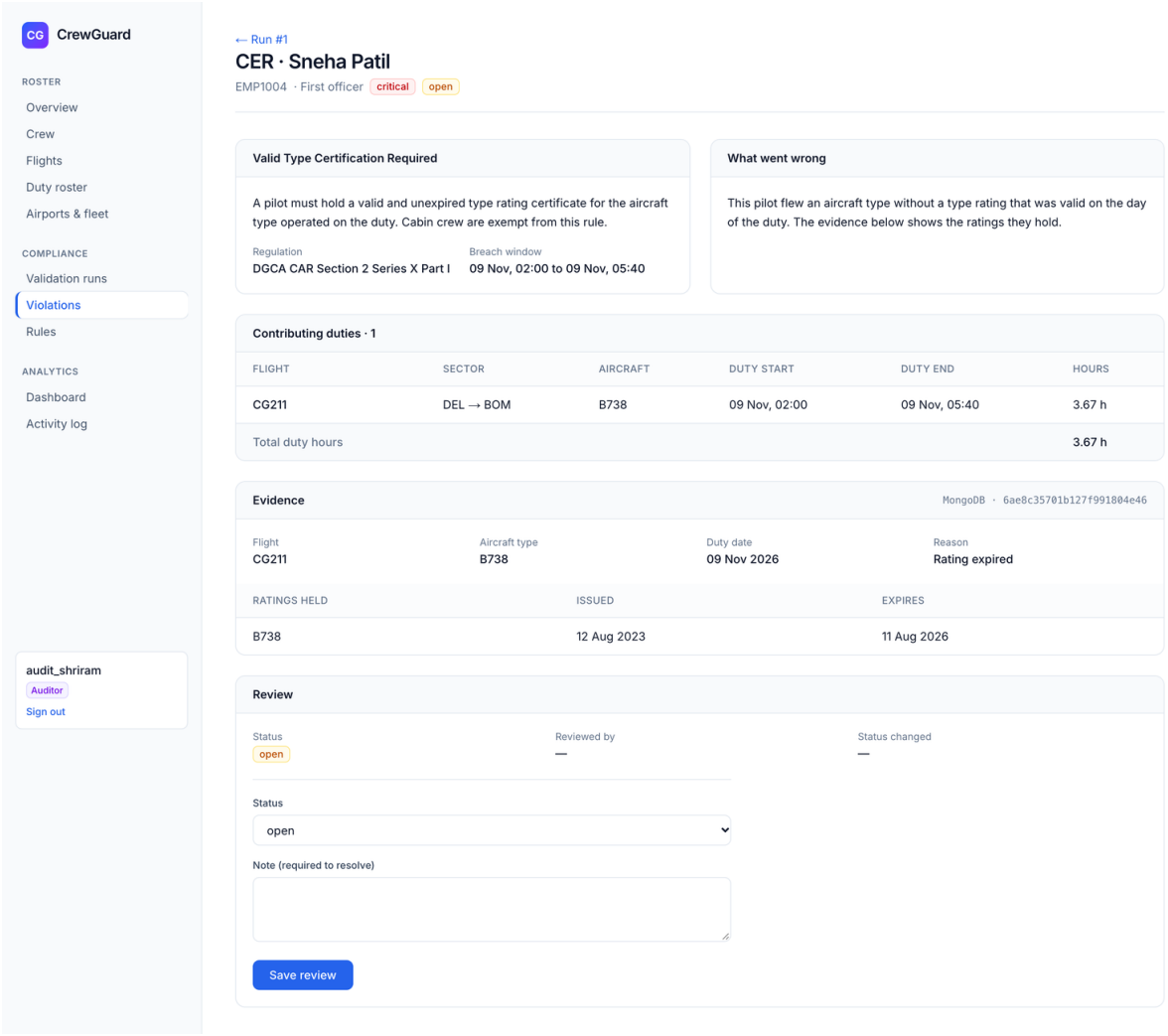


Figure 28: CER: Sneha flew a B738 on an expired rating.

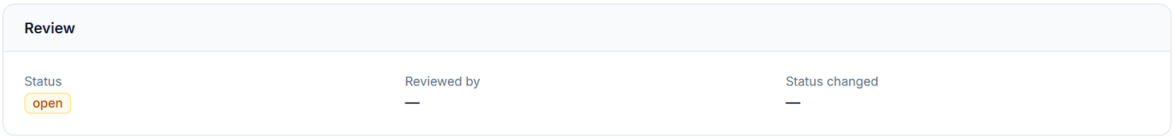


Figure 29: The Review card as the scheduler: read-only, with no form.

9.6 Rules and analytics

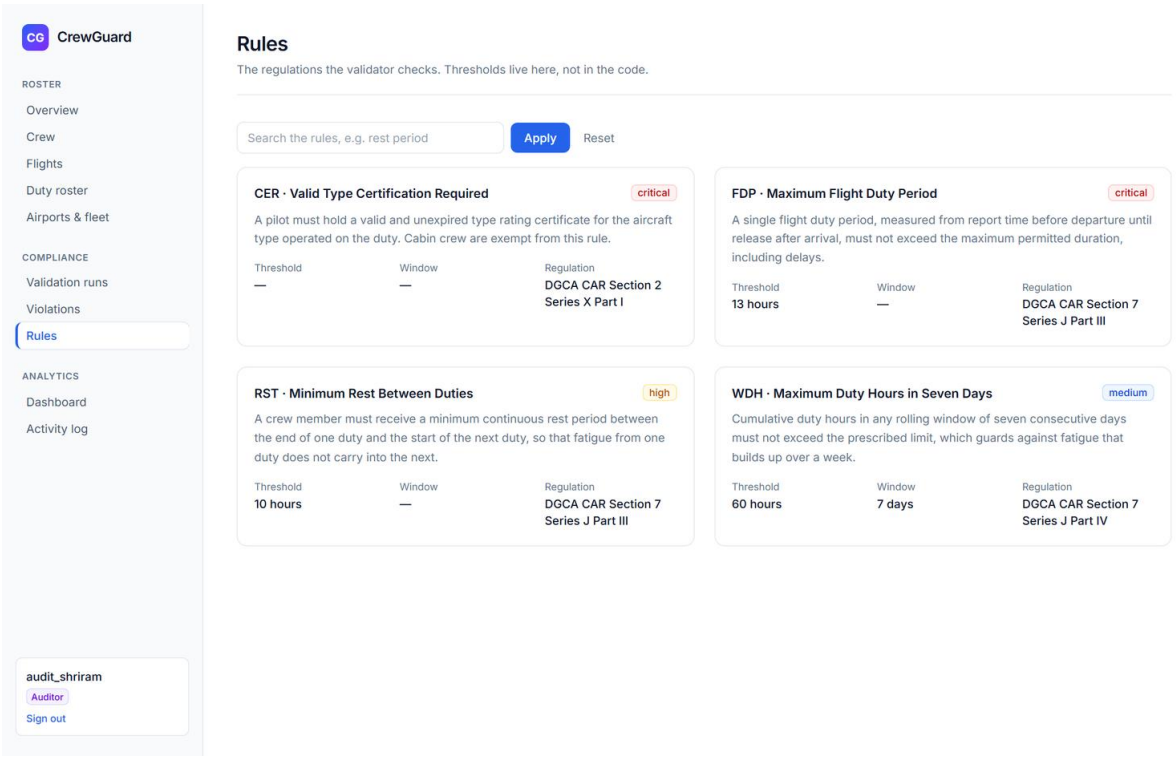


Figure 30: The rule catalogue.

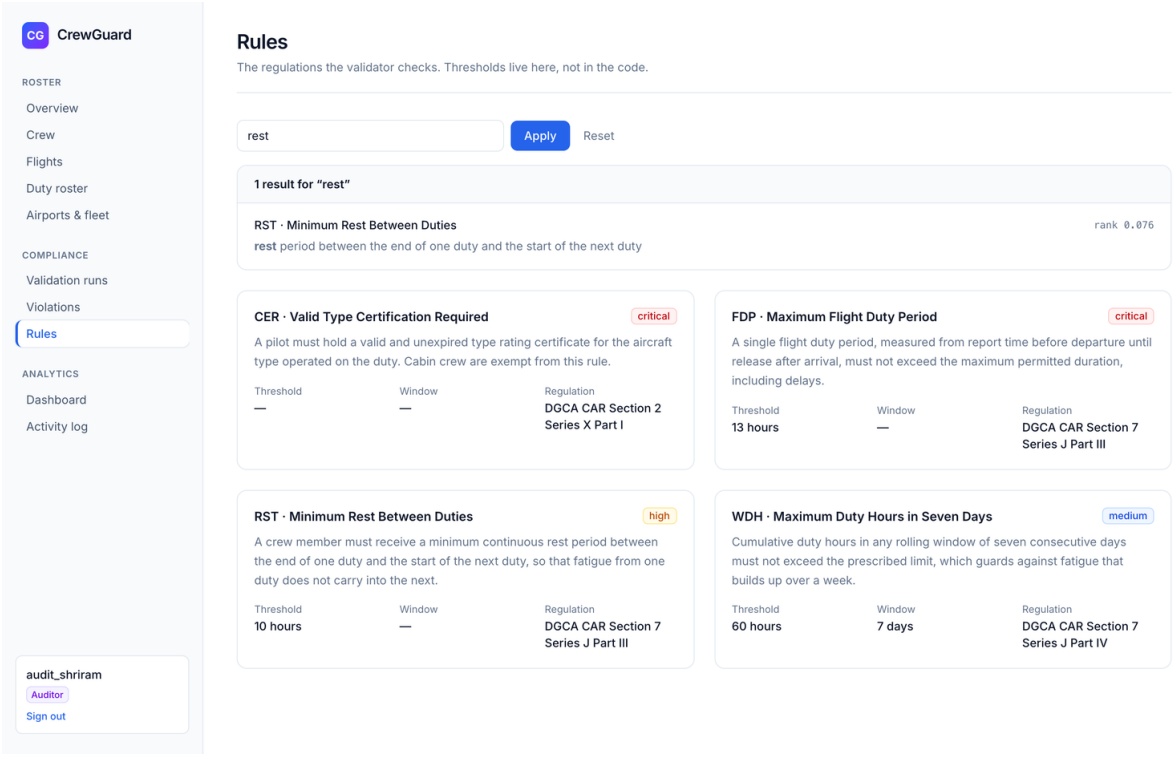


Figure 31: Full-text search for "rest", ranked, with the match highlighted.

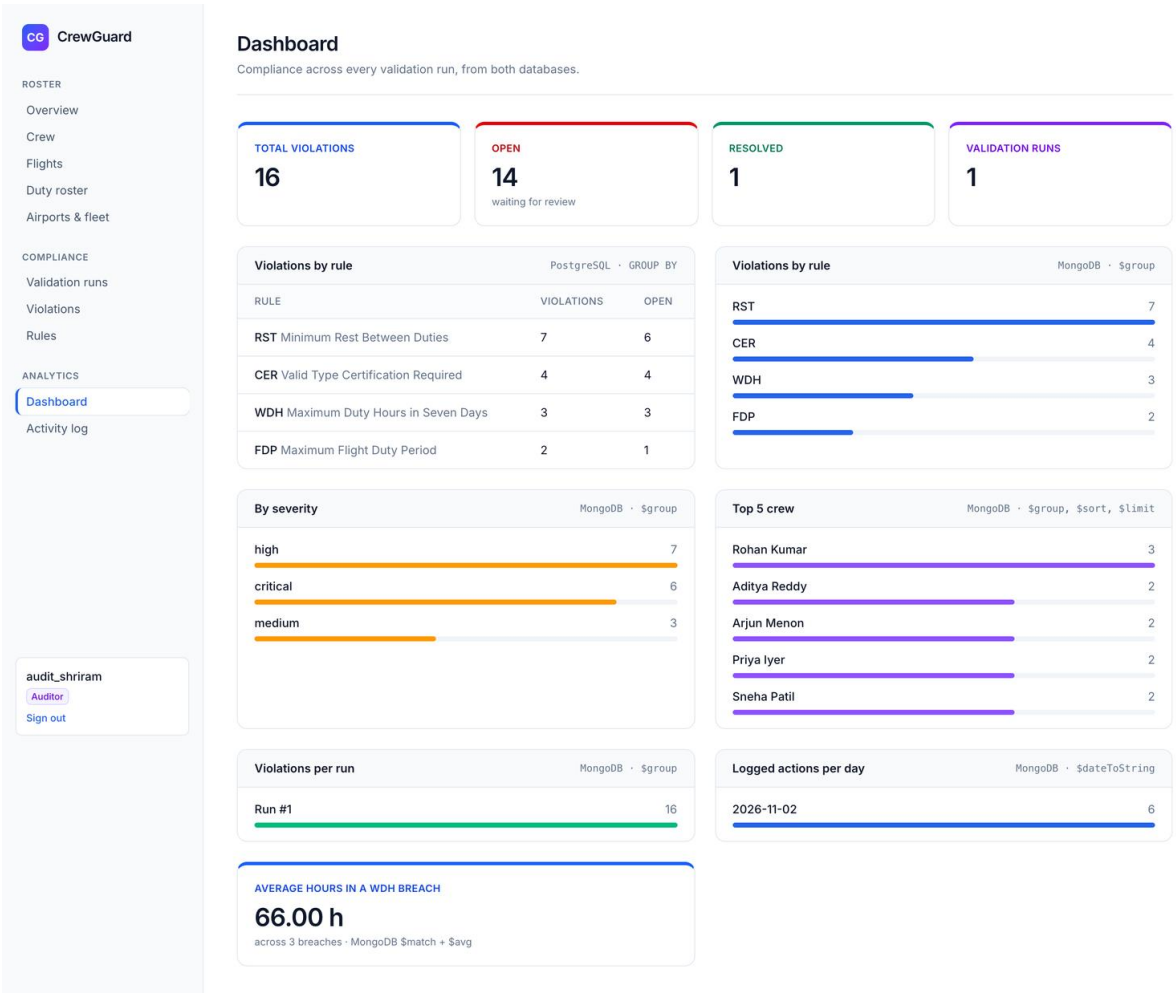


Figure 32: Dashboard: the same counts from a SQL GROUP BY and a MongoDB $group, side by side.

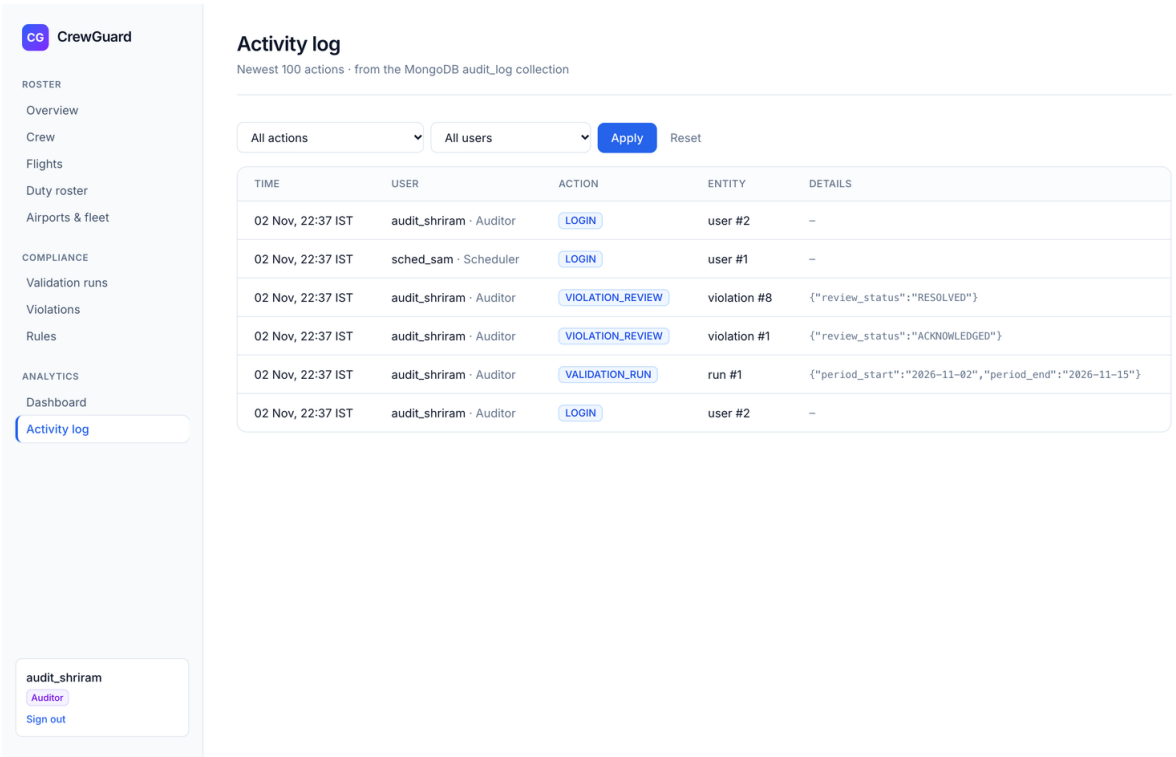


Figure 33: Activity log from the audit_log collection.

10. Testing

The automated tests use Node's built-in test runner against the real database. Every test runs inside `BEGIN ... ROLLBACK`, so the data is never changed and the tests can run at any time.

10.1 What the tests prove

Group	Tests	What it proves
Seed data	2	The sample data loads, and passwords are stored only as bcrypt hashes.
Constraints	10	UNIQUE, CHECK, FOREIGN KEY, EXCLUDE, ON DELETE RESTRICT and CASCADE are enforced by the database, each rejected by the expected constraint name.
trg_validate_duty	3	Inactive crew, a wrong role, and a duty not covering its flight are rejected.
Roster transaction	1	A failed statement rolls back the earlier ones as well.
Part 1 functions and views	6	The four functions and two views return the correct results.
Validator totals	6	The three periods give exactly the planted breaches. The result does not depend on the time zone, inactive rules are skipped, and thresholds come from the data.
The explanation	3	Each violation links exactly the duties that caused it, and WDH contributions add up to the total.
The traps	3	A renewed rating is not a breach, cabin crew are exempt, and the previous duty may lie before the period.
Compliance triggers	3	`violations_found`, `finished_at` and `reviewed_at` are maintained correctly.
Review and run CHECKs	3	An unknown status, RESOLVED without a note, and a reversed period are all rejected.
Validation transaction	1	Rolling back removes the run and everything it created.
v_violation_report	1	One row per violation, with the correct duty counts.
Full-text search	3	Ranking, stemming, and use of the GIN index.
MongoDB	1	A `$group` and a `$match` with `$avg` give the right results on a temporary collection.

10.2 Output of npm test

46 tests: 46 passed, 0 failed, 0 skipped. Run on the final code with MongoDB running (macOS, Node 26, PostgreSQL 16). Without MongoDB, its one test skips itself as designed.

```
✓ VALIDATOR: the full two weeks give RST 7, FDP 2, WDH 3, CER 4 (37.679166ms)
✓ VALIDATOR: week 1 gives RST 7, WDH 3, CER 2 (5.194166ms)
✓ VALIDATOR: week 2 gives FDP 2, CER 2 and no WDH (Nisha only reaches 58.67 h) (3.381834ms)
✓ VALIDATOR: the result does not depend on the session time zone (3.635625ms)
✓ VALIDATOR: an inactive rule is skipped (2.100583ms)
✓ VALIDATOR: thresholds come from rule_definition, not from the code (2.242875ms)
✓ EXPLANATION: Arjun's 0.25 h rest links exactly the two duties around it (2.962541ms)
✓ EXPLANATION: each WDH links 6 duties whose hours add up to 66 (2.330583ms)
✓ EXPLANATION: Aditya's FDP breach links the one delayed 14 h duty (2.190458ms)
✓ CER: Kavya holds a renewal, so her expired A320 certificate is not a breach (2.403292ms)
✓ CER: cabin crew are exempt, even with no ratings at all (2.683417ms)
✓ RST: the previous duty may lie before the period (0.870833ms)
✓ TRIGGER: violations_found follows inserts and deletes (2.737625ms)
✓ TRIGGER: finished_at is stamped when the run completes (7.278875ms)
✓ TRIGGER: reviewed_at is set when the review status changes, not by a note edit (2.341542ms)
✓ CHECK: unknown review status (2.099125ms)
✓ CHECK: a violation cannot be RESOLVED without a note (1.799417ms)
✓ CHECK: a run cannot end before it starts (0.399792ms)
✓ TRANSACTION: rolling back removes the run and everything it created (2.703584ms)
✓ VIEW v_violation_report gives one row per violation, with its duty count (2.761625ms)
✓ FULL-TEXT: searching 'rest' finds RST first (1.730334ms)
✓ FULL-TEXT: stemming matches other forms of a word (0.333292ms)
```

```
✓ FULL-TEXT: the search uses the GIN index (0.597333ms)
✓ seed data is loaded (20.363ms)
✓ passwords are stored as bcrypt hashes, never plain text (0.893833ms)
✓ UNIQUE: username already exists (2.164083ms)
✓ CHECK: IATA code must be three capital letters (1.072792ms)
✓ CHECK: unknown crew role (1.064042ms)
✓ FOREIGN KEY: flight with unknown aircraft type (1.163333ms)
✓ CHECK: flight cannot depart and arrive at the same airport (0.5975ms)
✓ CHECK: certificate must expire after it is issued (0.615542ms)
✓ UNIQUE: a crew member cannot be assigned twice to one flight (2.616833ms)
✓ EXCLUDE: a crew member cannot be on two overlapping duties (2.490625ms)
✓ ON DELETE RESTRICT: an airport used as a home base cannot be deleted (0.8225ms)
✓ ON DELETE CASCADE: deleting a crew member removes their duties and certificates (5.362667ms)
✓ TRIGGER: inactive crew cannot be rostered (0.652917ms)
✓ TRIGGER: assigned role must match the crew member's role (0.564417ms)
✓ TRIGGER: duty must cover the whole flight (0.611042ms)
✓ TRANSACTION: a failed statement rolls back the earlier ones too (0.999459ms)
✓ FUNCTION fn_duty_hours computes the duty length (0.387667ms)
✓ FUNCTION fn_crew_duty_hours sums a crew member's week (0.727166ms)
✓ FUNCTION fn_crew_roster returns one row per duty, in time order (0.793958ms)
✓ FUNCTION fn_cert_status classifies certificates (0.800583ms)
✓ VIEW v_crew_summary includes crew with no duties (LEFT JOIN) (0.715958ms)
✓ VIEW v_duty_roster joins every duty to its crew and flight (0.565833ms)
✓ MONGODB: $group counts documents per rule, $match + $avg reads a rule-specific field (43.87675ms)
ℹ tests 46
ℹ suites 0
ℹ pass 46
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 181.845209
```

10.3 Expected validator results

Period	RST	FDP	WDH	CER	Total
2 to 15 Nov 2026	7	2	3	4	16
2 to 8 Nov 2026	7	0	3	2	12
9 to 15 Nov 2026	0	2	0	2	4

10.4 End-to-end checks in the running app

Check	Result
Scheduler calls POST `/api/runs` or PATCH `/api/violations/:id`	403
POST `/api/runs` with no session	401
Run with reversed dates	400, `chk_run_period_order`
Run with a missing date	400, "Please fill in period start."
Resolve a violation with a blank note	400, `chk_violation_resolution_note`
Review with an unknown status	400, `chk_violation_review_status`
Open a violation or run that does not exist, or with a non-numeric id	404
Multi-assign with one overlapping crew member	400, nothing saved
Filters on the violations page by rule, severity, status, run and crew	Correct counts for each
Search text with quotes and symbols on the rules page	No error
Roster writes and validation runs while MongoDB is stopped	Succeed, and the pages say the evidence is unavailable

10.5 Tested against a real MongoDB

Run on macOS with Node 26, PostgreSQL 16 and MongoDB, on the final code. The type check, lint and production build were clean on the same code.

Check	Result
Validation for 2 to 15 Nov	16 violations (RST 7, FDP 2, WDH 3, CER 4), run COMPLETED, `violations_found` 16, `finished_at` set
Evidence documents	16 documents, and every `evidence_doc_id` resolves to the document with the same `violation_id`
Evidence card per rule	RST, FDP, WDH and CER (NO_RATING and EXPIRED) all show real data
Dashboard	MongoDB charts match the SQL table (7, 4, 3, 2). Severity 7 high, 6 critical, 3 medium. WDH average 66.00 h
Activity log	LOGIN, VALIDATION_RUN and VIOLATION_REVIEW entries for both users
`npm run db:reset`, then run again	Both collections emptied. New run gives 16 violations, all linked
A second run over 2 to 8 Nov	12 violations. Each run keeps its own evidence (16 and 12 grouped by `run_id`)

11. Conclusion and future enhancements

CrewGuard does what it set out to do. It keeps the roster in a strongly constrained relational database, finds exactly the breaches planted in the sample data with one stored procedure, and explains each one down to the duties that caused it. Keeping evidence and the activity log in MongoDB suited their shape, and treating PostgreSQL as the source of truth means losing MongoDB costs some information on screen, but no compliance data.

The main thing we would point out is the difference between prevention and detection. Constraints stop what is wrong in one row or one pair of rows. The rules that matter most here only show up across many legal-looking rows, which needs window functions and a validator.

Known limitations

- The two databases are not kept in step after a run. If a crew member is deleted, their violations go by cascade but their evidence documents stay, so the MongoDB charts can run ahead of the SQL panel until the next reset.
- A run is a snapshot. Changing the roster afterwards does not change an earlier run, and the `STALE` status exists in the schema but is not set by anything yet.
- The rules are simplified versions of the regulation.

Future enhancements

- More rules, such as a 28-day duty limit and a weekly day off. These were in the Review 2 catalogue but were dropped because nothing implemented them.
- Mark earlier runs as `STALE` when a duty in their period changes.
- Remove or flag evidence documents when their violation is deleted.
- Vector search over the full regulation text, in addition to full-text search on the rule catalogue.

12. Individual contributions

Area	Samyak Sanklecha (PES2UG24CS436), Part 1	Shriram Chandrasekar (PES2UG24CS495), Part 2
PostgreSQL	Schema with 11 tables, constraints including EXCLUDE, and indexes. `db/02_roster.sql`: functions, views and `trg_validate_duty`. Seed data. Queries Q1 to Q12	`db/03_compliance.sql`: review columns with ALTER, three triggers, `v_violation_report`, `sp_run_validation`, full-text search
MongoDB	None	`violation_evidence` and `audit_log`, aggregation pipelines
Application	Backend to database (pool, parameterised queries, `withTransaction`), auth and roles, roster UI, UI theme	Compliance and analytics UI
Quality	Database constraint tests	Compliance and MongoDB tests, diagrams, `docs/DEMO.md`, this report